

Linux 系统基础

关于 Linux 操作系统的基础知识

王少东

南京工程学院通信工程学院

二〇二〇年四月

Part I

Linux 简介

引子

- Linux 是什么？
- 这门课讲什么？
- 为什么学习 Linux？
- 能够从这门课中得到什么？
- 如何学习 Linux？
- 这门课如何安排？

Linux

- Linux 是一种计算机操作系统
- 就像 Windows, MacOS 一样
- 应用于服务器、个人电脑、嵌入式系统
- 从 1992 年前后, 逐渐流行
- 是一种自由/开源软件

Linux 与 UNIX

- UNIX 是诞生于 1969 年的“老牌”操作系统
- 最初属于美国电话电报公司贝尔实验室
- Windows、MacOS、Linux 都深受 UNIX 系统影响
- Linux 被称为“类 UNIX 操作系统”

类 Unix 操作系统

人们有时会用 Unix 表示一类操作系统，这其中包括 Linux。有时，我们会把这类操作系统称为“类 UNIX 操作系统”。

- AIX (IBM)
- HP-UX (HP)
- Solaris (SUN, Oracle)
- FreeBSD, NetBSD, OpenBSD (Berkeley Software Distribution)
- GNU/Linux

Linux 与 嵌入式系统

- 相当一部分嵌入式设备，采用 Linux 操作系统
- Linux 并不仅仅应用于嵌入式设备
- Linux 在服务器和桌面系统上都拥有很高的占有率
- 事实上，Linux 首先是在服务器上的良好表现，开始慢慢被公众接受的。

Linux 系统的两类用户

- 两类主要的用户

- 普通用户：使用 Linux 系统上的应用程序和 Linux 系统的基本功能
- 程序员：应用程序开发者。使用系统提供的编程接口

- 两个主要的接口

- 命令与实用工具
- 系统调用与库函数

Linux 系统的两类用户

- 两类主要的用户
 - 普通用户：使用 Linux 系统上的应用程序和 Linux 系统的基本功能
 - 程序员：应用程序开发者。使用系统提供的编程接口
- 两个主要的接口
 - 命令与实用工具
 - 系统调用与库函数

Linux 系统的两类用户

- 两类主要的用户
 - 普通用户：使用 Linux 系统上的应用程序和 Linux 系统的基本功能
 - 程序员：应用程序开发者。使用系统提供的编程接口
- 两个主要的接口
 - 命令与实用工具
 - 系统调用与库函数

课程内容

- 学习 Linux 环境中的基本概念
- 使用 Linux 的常用命令和使用工具
- 理解 Linux 的文件系统
- Linux 系统中的用户、组、权限
- 学习 shell（重定向，管道，文件名替换）
- 了解正则表达式
- 掌握一种文本编辑工具

学习动机

- Linux 是最经典的操作系统
- 广泛应用于各种领域
- 富有特色的命令和实用工具集合
- Linux 是程序员的操作系统
- 拥有独特的技术魅力
- 深远的技术文化影响

学习方法

- 注意总结规律
- 重视实验和练习
- 增加课外训练
- 勇于尝试
- 注重讨论和交流

实践最重要

唯有使用 Linux ， 才能学会 Linux 。
别无他法。

授课计划

- 讲授：24 学时
- 实验：8 学时
- 建议的课外练习：不少于 32 学时

建立自己的练习环境

- 获得 Live CD 的镜像，并写到 CDROM 光盘上。
- 制作一只可以启动的 USB 盘。（或者移动硬盘）
- 在 Windows 操作系统上，安装虚拟机软件（vmware, VirtualPC, VirtualBox），在虚拟机上安装 Linux 系统。
- 在 Windows 分区上，安装一套独立的 Linux 系统。（整个 Linux 系统，作为一个 Windows 的文件。wubi）
- 调整硬盘的分区，在空闲的分区上，安装 Linux 操作系统。（双系统）
- 在整个硬盘上，安装 Linux 系统。（为了偶尔使用 Windows，可以在 Linux 系统上安装虚拟机，在虚拟机当中，安装 Windows 系统。）

建立自己的练习环境

- 获得 Live CD 的镜像，并写到 CDROM 光盘上。
- 制作一只可以启动的 USB 盘。（或者移动硬盘）
- 在 Windows 操作系统上，安装虚拟机软件（vmware, VirtualPC, VirtualBox），在虚拟机上安装 Linux 系统。
- 在 Windows 分区上，安装一套独立的 Linux 系统。（整个 Linux 系统，作为一个 Windows 的文件。wubi）
- 调整硬盘的分区，在空闲的分区上，安装 Linux 操作系统。（双系统）
- 在整个硬盘上，安装 Linux 系统。（为了偶尔使用 Windows，可以在 Linux 系统上安装虚拟机，在虚拟机当中，安装 Windows 系统。）

建立自己的练习环境

- 获得 Live CD 的镜像，并写到 CDROM 光盘上。
- 制作一只可以启动的 USB 盘。（或者移动硬盘）
- 在 Windows 操作系统上，安装虚拟机软件（vmware, VirtualPC, VirtualBox），在虚拟机上安装 Linux 系统。
- 在 Windows 分区上，安装一套独立的 Linux 系统。（整个 Linux 系统，作为一个 Windows 的文件。wubi）
- 调整硬盘的分区，在空闲的分区上，安装 Linux 操作系统。（双系统）
- 在整个硬盘上，安装 Linux 系统。（为了偶尔使用 Windows，可以在 Linux 系统上安装虚拟机，在虚拟机当中，安装 Windows 系统。）

建立自己的练习环境

- 获得 Live CD 的镜像，并写到 CDROM 光盘上。
- 制作一只可以启动的 USB 盘。（或者移动硬盘）
- 在 Windows 操作系统上，安装虚拟机软件（vmware, VirtualPC, VirtualBox），在虚拟机上安装 Linux 系统。
- 在 Windows 分区上，安装一套独立的 Linux 系统。（整个 Linux 系统，作为一个 Windows 的文件。wubi）
- 调整硬盘的分区，在空闲的分区上，安装 Linux 操作系统。（双系统）
- 在整个硬盘上，安装 Linux 系统。（为了偶尔使用 Windows，可以在 Linux 系统上安装虚拟机，在虚拟机当中，安装 Windows 系统。）

建立自己的练习环境

- 获得 Live CD 的镜像，并写到 CDROM 光盘上。
- 制作一只可以启动的 USB 盘。（或者移动硬盘）
- 在 Windows 操作系统上，安装虚拟机软件（vmware, VirtualPC, VirtualBox），在虚拟机上安装 Linux 系统。
- 在 Windows 分区上，安装一套独立的 Linux 系统。（整个 Linux 系统，作为一个 Windows 的文件。wubi）
- 调整硬盘的分区，在空闲的分区上，安装 Linux 操作系统。（双系统）
- 在整个硬盘上，安装 Linux 系统。（为了偶尔使用 Windows，可以在 Linux 系统上安装虚拟机，在虚拟机当中，安装 Windows 系统。）

建立自己的练习环境

- 获得 Live CD 的镜像，并写到 CDROM 光盘上。
- 制作一只可以启动的 USB 盘。（或者移动硬盘）
- 在 Windows 操作系统上，安装虚拟机软件（vmware, VirtualPC, VirtualBox），在虚拟机上安装 Linux 系统。
- 在 Windows 分区上，安装一套独立的 Linux 系统。（整个 Linux 系统，作为一个 Windows 的文件。wubi）
- 调整硬盘的分区，在空闲的分区上，安装 Linux 操作系统。（双系统）
- 在整个硬盘上，安装 Linux 系统。（为了偶尔使用 Windows，可以在 Linux 系统上安装虚拟机，在虚拟机当中，安装 Windows 系统。）

Windows 10 系统用户

- 在最新的 Windows 10 系统中，增加了 Linux 子系统
- Windows Subsystem for Linux
- 启用该模块之后，就可以直接在 Windows 系统里面获得 Linux 环境

Windows 10 系统用户

- 在最新的 Windows 10 系统中，增加了 Linux 子系统
- Windows Subsystem for Linux
- 启用该模块之后，就可以直接在 Windows 系统里面获得 Linux 环境

Windows 10 系统用户

- 在最新的 Windows 10 系统中，增加了 Linux 子系统
- Windows Subsystem for Linux
- 启用该模块之后，就可以直接在 Windows 系统里面获得 Linux 环境

使用苹果电脑（MacOS 系统）的用户

- MacOS 本身包含一个“终端”应用
- Terminal
- 启动该应用之后，得到一个窗口
- 终端窗口内就可以使用 Linux 命令

使用苹果电脑（MacOS 系统）的用户

- MacOS 本身包含一个“终端”应用
- Terminal
- 启动该应用之后，得到一个窗口
- 终端窗口内就可以使用 Linux 命令

使用苹果电脑（MacOS 系统）的用户

- MacOS 本身包含一个“终端”应用
- Terminal
- 启动该应用之后，得到一个窗口
- 终端窗口内就可以使用 Linux 命令

使用苹果电脑（MacOS 系统）的用户

- MacOS 本身包含一个“终端”应用
- Terminal
- 启动该应用之后，得到一个窗口
- 终端窗口内就可以使用 Linux 命令

在浏览器中运行的仿真 Linux 系统

- <https://bellard.org/jslinux>
- 选择一个 User Interface 类型为 Console 的链接进去
- 立刻得到一个可以运行 Linux 命令的环境
- 其他：
- <http://www.webminal.org/>
- <http://www.tutorialspoint.com/codingground.htm>
- <https://linuxzoo.net/>

在浏览器中运行的仿真 Linux 系统

- <https://bellard.org/jslinux>
- 选择一个 User Interface 类型为 Console 的链接进去
- 立刻得到一个可以运行 Linux 命令的环境
- 其他：
- <http://www.webminal.org/>
- <http://www.tutorialspoint.com/codingground.htm>
- <https://linuxzoo.net/>

在浏览器中运行的仿真 Linux 系统

- <https://bellard.org/jslinux>
- 选择一个 User Interface 类型为 Console 的链接进去
- 立刻得到一个可以运行 Linux 命令的环境
- 其他：
- <http://www.webminal.org/>
- <http://www.tutorialspoint.com/codingground.htm>
- <https://linuxzoo.net/>

在浏览器中运行的仿真 Linux 系统

- <https://bellard.org/jslinux>
- 选择一个 User Interface 类型为 Console 的链接进去
- 立刻得到一个可以运行 Linux 命令的环境
- 其他：
 - <http://www.webminal.org/>
 - <http://www.tutorialspoint.com/codingground.htm>
 - <https://linuxzoo.net/>

在浏览器中运行的仿真 Linux 系统

- <https://bellard.org/jslinux>
- 选择一个 User Interface 类型为 Console 的链接进去
- 立刻得到一个可以运行 Linux 命令的环境
- 其他：
- <http://www.webminal.org/>
- <http://www.tutorialspoint.com/codingground.htm>
- <https://linuxzoo.net/>

在浏览器中运行的仿真 Linux 系统

- <https://bellard.org/jslinux>
- 选择一个 User Interface 类型为 Console 的链接进去
- 立刻得到一个可以运行 Linux 命令的环境
- 其他：
- <http://www.webminal.org/>
- <http://www.tutorialspoint.com/codingground.htm>
- <https://linuxzoo.net/>

在浏览器中运行的仿真 Linux 系统

- <https://bellard.org/jslinux>
- 选择一个 User Interface 类型为 Console 的链接进去
- 立刻得到一个可以运行 Linux 命令的环境
- 其他：
- <http://www.webminal.org/>
- <http://www.tutorialspoint.com/codingground.htm>
- <https://linuxzoo.net/>

发行版本的选择

- 当你去安装一个 Linux 系统时……
- 要选择具体的“发行版”
- Ubuntu, Fedora, Debian, SuSe ……等等
- 参考：distrowatch.com
- 我推荐的发行版 MXLinux, mint, ubuntu, fedora
- Manjaro? deepin?

发行版本的选择

- 当你去安装一个 Linux 系统时……
- 要选择具体的“发行版”
- Ubuntu, Fedora, Debian, SuSe ……等等
- 参考：distrowatch.com
- 我推荐的发行版 MXLinux, mint, ubuntu, fedora
- Manjaro? deepin?

发行版本的选择

- 当你去安装一个 Linux 系统时……
- 要选择具体的“发行版”
- Ubuntu, Fedora, Debian, SuSe ……等等
- 参考：distrowatch.com
- 我推荐的发行版 MXLinux, mint, ubuntu, fedora
- Manjaro? deepin?

发行版本的选择

- 当你去安装一个 Linux 系统时……
- 要选择具体的“发行版”
- Ubuntu, Fedora, Debian, SuSe ……等等
- 参考：distrowatch.com
- 我推荐的发行版 MXLinux, mint, ubuntu, fedora
- Manjaro? deepin?

发行版本的选择

- 当你去安装一个 Linux 系统时……
- 要选择具体的“发行版”
- Ubuntu, Fedora, Debian, SuSe ……等等
- 参考：distrowatch.com
- 我推荐的发行版 MXLinux, mint, ubuntu, fedora
- Manjaro? deepin?

发行版本的选择

- 当你去安装一个 Linux 系统时……
- 要选择具体的“发行版”
- Ubuntu, Fedora, Debian, SuSe ……等等
- 参考：distrowatch.com
- 我推荐的发行版 MXLinux, mint, ubuntu, fedora
- Manjaro? deepin?

操作系统

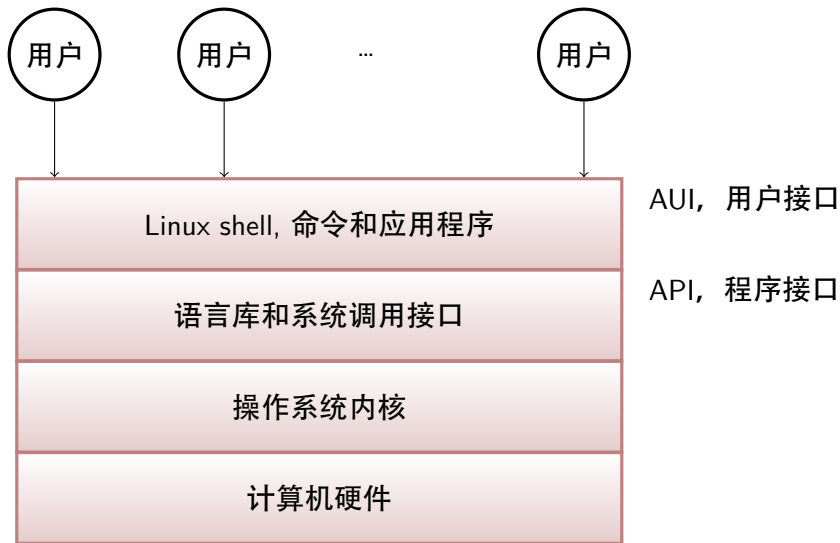
操作系统是负责管理计算机系统软硬件资源，使用户简单、高效、公平、有序、安全地使用计算机系统的系统软件。

- 为用户和应用程序提供接口
- 管理并分配资源
- 加载并执行用户程序

- 两类主要的用户：一般用户，程序员
- 两个主要的接口：AUI, API
 - Application User's Interface
 - Application Programmer's Interface
- 四个层次
 - Linux shell, 命令和应用程序
 - 语言库和系统调用接口
 - 操作系统内核
 - 计算机硬件

- 两类主要的用户：一般用户，程序员
- 两个主要的接口：AUI, API
 - Application User's Interface
 - Application Programmer's Interface
- 四个层次
 - Linux shell, 命令和应用程序
 - 语言库和系统调用接口
 - 操作系统内核
 - 计算机硬件

- 两类主要的用户：一般用户，程序员
- 两个主要的接口：AUI, API
 - Application User's Interface
 - Application Programmer's Interface
- 四个层次
 - Linux shell, 命令和应用程序
 - 语言库和系统调用接口
 - 操作系统内核
 - 计算机硬件



两种类型的用户界面：

- 字符用户界面，CUI
- 图形用户界面，GUI

CUI, Character User Interface

GUI, Graphical User Interface

两种类型的用户界面：

- 字符用户界面，CUI
- 图形用户界面，GUI

CUI, Character User Interface

GUI, Graphical User Interface

Linux 的特点

- 多用户
- 多任务
- 层次型文件系统
- I / O 操作与设备无关
- 独特的用户界面：shell
- 丰富而完善的系统工具
- 出色的服务功能
- 支持多种硬件平台（Linux 是用 C 语言编写的）

Linux 的基本结构

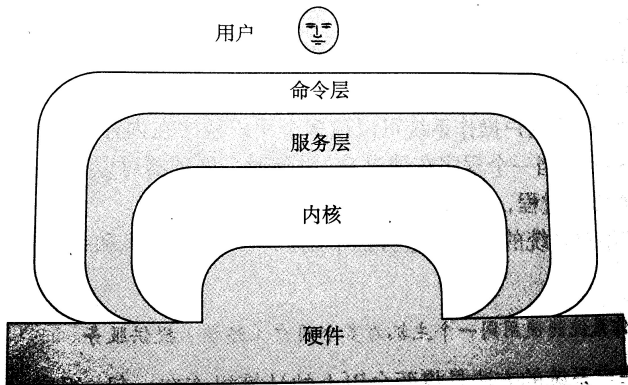


图 1.6 操作系统的分层结构

作业

在你自己的计算机上安装一套 Linux
完成作业一

Part II

Linux 系统入门

登录

Linux 是一个多用户系统，使用 Linux 系统必须首先登录。

- 系统在终端上显示登录提示：

Login:

- 输入用户名，回车结束
- 输口令，回车结束
(口令不会显示在屏幕上，连星号也不显示)
- 验证通过后，显示系统信息，并出现系统提示符。
\$ 或者% 或者 #

退出

- 在系统提示符下，按 Ctrl-D 组合键（不用回车）
- 或者，输入 exit 命令（需要回车）

工作结束后及时退出系统的意义

- 防止他人冒用系统
- 停止计费
- 释放该用户占用的资源

退出

- 在系统提示符下，按 Ctrl-D 组合键（不用回车）
- 或者，输入 exit 命令（需要回车）

工作结束后及时退出系统的意义

- 防止他人冒用系统
- 停止计费
- 释放该用户占用的资源

退出

- 在系统提示符下，按 Ctrl-D 组合键（不用回车）
- 或者，输入 exit 命令（需要回车）

工作结束后及时退出系统的意义

- 防止他人冒用系统
- 停止计费
- 释放该用户占用的资源

会话 (Session)

- 用户使用一次计算机的全过程
- 从登录开始
- 到退出结束

口令

- 修改口令的命令：
passwd
- 普通用户：
只能修改自己的口令
口令太短或者太简单，系统可能拒绝修改
- 超级用户：
有权修改任何人的口令
对口令的指定，没有更多限制

口令

- 修改口令的命令：
passwd
- 普通用户：
只能修改自己的口令
口令太短或者太简单，系统可能拒绝修改
- 超级用户：
有权修改任何人的口令
对口令的指定，没有更多限制

口令

- 修改口令的命令：
passwd
- 普通用户：
只能修改自己的口令
口令太短或者太简单，系统可能拒绝修改
- 超级用户：
有权修改任何人的口令
对口令的指定，没有更多限制

用户

- 普通用户（一般用户）
权限受限
- 超级用户（系统管理员，根用户，root 用户）
权限不受限制
- 超级用户的提示符，与普通用户的提示符有区别
#（井号）提示符，专门用于超级用户，提醒用户注意
- 切换用户身份的 Linux 命令：
su [< 用户名 >]
- 普通用户临时需要执行特权命令，可以使用：
sudo < 命令 >

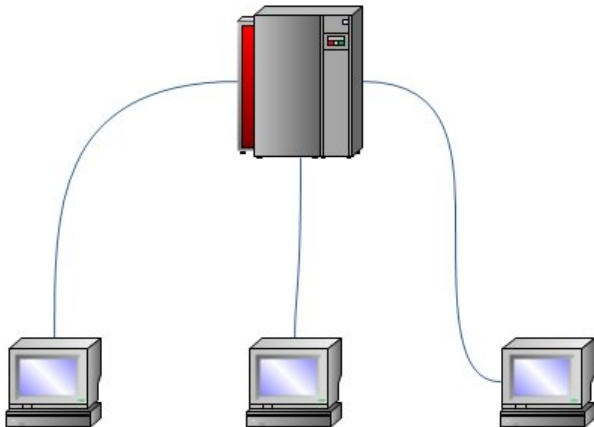
用户

- 普通用户（一般用户）
权限受限
- 超级用户（系统管理员，根用户，root 用户）
权限不受限制
- 超级用户的提示符，与普通用户的提示符有区别
#（井号）提示符，专门用于超级用户，提醒用户注意
- 切换用户身份的 Linux 命令：
su [< 用户名 >]
- 普通用户临时需要执行特权命令，可以使用：
sudo < 命令 >

用户

- 普通用户（一般用户）
权限受限
- 超级用户（系统管理员，根用户，root 用户）
权限不受限制
- 超级用户的提示符，与普通用户的提示符有区别
#（井号）提示符，专门用于超级用户，提醒用户注意
- 切换用户身份的 Linux 命令：
su [< 用户名 >]
- 普通用户临时需要执行特权命令，可以使用：
sudo < 命令 >

主机终端模式



Linux PC 上的虚拟终端



Linux PC 上的虚拟终端



Linux 环境下的 12 个虚拟终端（控制台）：

- 字符终端

Ctrl + Alt + F1

Ctrl + Alt + F2

...

Ctrl + Alt + F6

- 图形终端

Ctrl + Alt + F7

Ctrl + Alt + F8

...

Ctrl + Alt + F12

终端和命令行

- Linux 系统通过在终端上输入命令操作计算机。
- 所输入的命令，用回车符作为结束。这被称为命令行
- 命令行是逻辑上的行，不受屏幕宽度的限制
- 行尾反斜线符号 “\”，表示命令行未完，后面还有续行

命令行格式

\$ <命令名> [-<选项>] [<参数>] <回车>

- 系统提示符 提示用户可以输入命令
- 命令名 (大小写敏感)
- 选项 (减号, 或两个减号引导)
- 参数 (多个参数用空格分隔)
- 回车符作为结束

命令行举例

```
$ ls <回车>  
$ ls  
ls
```

```
ls music  
ls -l music
```

```
ls -l -a music  
ls -la music
```

```
ls --format=long --all music
```

```
ls --format=lo\  
> ng --all music
```

命令行举例

```
$ ls <回车>  
$ ls  
ls
```

```
ls music  
ls -l music
```

```
ls -l -a music  
ls -la music
```

```
ls --format=long --all music
```

```
ls --format=lo\  
> ng --all music
```

命令行举例

```
$ ls <回车>  
$ ls  
ls
```

```
ls music  
ls -l music
```

```
ls -l -a music  
ls -la music
```

```
ls --format=long --all music
```

```
ls --format=lo\  
> ng --all music
```


命令行举例

```
$ ls <回车>
$ ls
ls
```

```
ls music
ls -l music
```

```
ls -l -a music
ls -la music
```

```
ls --format=long --all music
```

```
ls --format=lo\
> ng --all music
```

命令行举例

```
$ ls <回车>  
$ ls  
ls
```

```
ls music  
ls -l music
```

```
ls -l -a music  
ls -la music
```

```
ls --format=long --all music
```

```
ls --format=lo\  
> ng --all music
```

几条简单的 Linux 命令

- 时间和日期命令：

```
date
```

- 用户信息显示：

```
who
```

- 显示日历：

```
cal
```

几条简单的 Linux 命令

- 时间和日期命令：

```
date
```

- 用户信息显示：

```
who
```

- 显示日历：

```
cal
```

几条简单的 Linux 命令

- 时间和日期命令：

```
date
```

- 用户信息显示：

```
who
```

- 显示日历：

```
cal
```

命令的变化：选项

```
who -H  
who --heading
```

```
cal 10 2012  
cal -m 2  
cal -3  
cal -3 -m 2
```

哪些选项？

命令的变化：选项

```
who -H  
who --heading
```

```
cal 10 2012  
cal -m 2  
cal -3  
cal -3 -m 2
```

哪些选项？

命令的变化：选项

```
who -H  
who --heading
```

```
cal 10 2012  
cal -m 2  
cal -3  
cal -3 -m 2
```

哪些选项？

联机手册

- Linux 系统中一般都安装有用户手册的电子版。
查看联机手册的命令：

```
man
```

- 用法

```
man <命令名>
```

- 例子

```
man cal  
man date  
man who
```

- 联机手册分为若干章节，可用以下命令进一步了解：

```
man man
```

- 要退出联机帮助，按小写的“q”键。

联机手册

- Linux 系统中一般都安装有用户手册的电子版。
查看联机手册的命令：

```
man
```

- 用法

```
man <命令名>
```

- 例子

```
man cal  
man date  
man who
```

- 联机手册分为若干章节，可用以下命令进一步了解：

```
man man
```

- 要退出联机帮助，按小写的“q”键。

联机手册

- Linux 系统中一般都安装有用户手册的电子版。
查看联机手册的命令：

```
man
```

- 用法

```
man <命令名>
```

- 例子

```
man cal  
man date  
man who
```

- 联机手册分为若干章节，可用以下命令进一步了解：

```
man man
```

- 要退出联机帮助，按小写的“q”键。

联机手册

- Linux 系统中一般都安装有用户手册的电子版。
查看联机手册的命令：

```
man
```

- 用法

```
man <命令名>
```

- 例子

```
man cal  
man date  
man who
```

- 联机手册分为若干章节，可用以下命令进一步了解：

```
man man
```

- 要退出联机帮助，按小写的“q”键。

联机手册

- Linux 系统中一般都安装有用户手册的电子版。
查看联机手册的命令：

```
man
```

- 用法

```
man <命令名>
```

- 例子

```
man cal  
man date  
man who
```

- 联机手册分为若干章节，可用以下命令进一步了解：

```
man man
```

- 要退出联机帮助，按小写的“q”键。

Linux 的联机手册分为若干“节” (section)：

- 1 Executable programs or shell commands
- 2 System calls (functions provided by the kernel)
- 3 Library calls (functions within program libraries)
- 4 Special files (usually found in /dev)
- 5 File formats and conventions eg /etc/passwd
- 6 Games
- 7 Miscellaneous (including macro packages
and conventions), e.g. man(7), groff(7)
- 8 System administration commands (usually only for root)
- 9 Kernel routines [Non standard]

根据关键词查联机手册

- 不知道命令名，如何查找信息？
可以使用关键字查找：

```
man -k <keyword>
```

- 例子

```
man -k calendar  
man -k line number
```

- 输出信息太多，难以辨认时，可采用管道和过滤命令筛选：

```
man -k line number | grep count
```

根据关键词查联机手册

- 不知道命令名，如何查找信息？
可以使用关键字查找：

```
man -k <keyword>
```

- 例子

```
man -k calendar  
man -k line number
```

- 输出信息太多，难以辨认时，可采用管道和过滤命令筛选：

```
man -k line number | grep count
```


根据关键词查联机手册

- 不知道命令名，如何查找信息？
可以使用关键字查找：

```
man -k <keyword>
```

- 例子

```
man -k calendar  
man -k line number
```

- 输出信息太多，难以辨认时，可采用管道和过滤命令筛选：

```
man -k line number | grep count
```

其他获得帮助的渠道

- 教学程序 learn（如果系统没有安装该程序，系统将会显示：
bash: learn: command not found 之类的信息）
- 一般性帮助 help（系统之间不同）
- 联机文档：info
- apropos 命令
- whatis 命令
- 在命令之后使用帮助选项
--help, --usage, -?, -h 等。

纠正错误：命令行编辑

- 删除光标左侧的一个字符
[Backspace] 键，或者 [Ctrl - h] 组合键
- 删除一整行
[Ctrl - u] 组合键
(删除光标左侧的所有内容)
- 其他更多的编辑命令键
[Ctrl - w], [Ctrl - k], ...
- 提示：不要强记这些键的功能，注意掌握概念。

纠正错误：命令行编辑

- 删除光标左侧的一个字符
[Backspace] 键，或者 [Ctrl - h] 组合键
- 删除一整行
[Ctrl - u] 组合键
(删除光标左侧的所有内容)
- 其他更多的编辑命令键
[Ctrl - w], [Ctrl - k], ...
- 提示：不要强记这些键的功能，注意掌握概念。

纠正错误：命令行编辑

- 删除光标左侧的一个字符
[Backspace] 键，或者 [Ctrl - h] 组合键
- 删除一整行
[Ctrl - u] 组合键
(删除光标左侧的所有内容)
- 其他更多的编辑命令键
[Ctrl - w], [Ctrl - k], ...
- 提示：不要强记这些键的功能，注意掌握概念。

纠正错误：命令行编辑

- 删除光标左侧的一个字符
[Backspace] 键，或者 [Ctrl - h] 组合键
- 删除一整行
[Ctrl - u] 组合键
(删除光标左侧的所有内容)
- 其他更多的编辑命令键
[Ctrl - w], [Ctrl - k], ...
- 提示：不要强记这些键的功能，注意掌握概念。

小结

- 登录与退出
- 命令行与命令行编辑
- 简单的 Linux 命令
who, date, cal
- 帮助

Part III

Linux 文件系统基础

情境

Tom 与 David 合作完成一个项目。

Tom 写了一个便条，保存成文件，给 David 看。

- ① 文件的名字
- ② 保存在什么地方
- ③ 文件的格式
- ④ 如何查看
- ⑤ 如何修改、改名、移动、删除

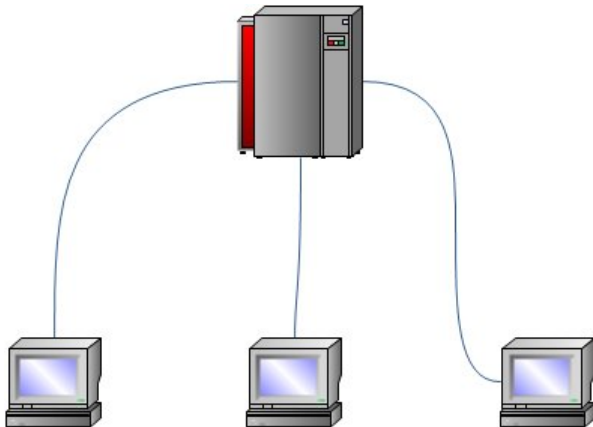
情境

Tom 与 David 合作完成一个项目。

Tom 写了一个便条，保存成文件，给 David 看。

- ① 文件的名字
- ② 保存在什么地方
- ③ 文件的格式
- ④ 如何查看
- ⑤ 如何修改、改名、移动、删除

主机终端模式



文件和目录

- 文件的一般概念：
文件是对外存储器上相关数据集合的抽象。
- Linux 扩展了文件的概念：
In Linux Everything is a File
- 目录系统提供了组织文件的结构：
目录也是一种文件

文件和目录

- 文件的一般概念：
文件是对外存储器上相关数据集合的抽象。
- Linux 扩展了文件的概念：
In Linux Everything is a File
- 目录系统提供了组织文件的结构：
目录也是一种文件

文件和目录

- 文件的一般概念：
文件是对外存储器上相关数据集合的抽象。
- Linux 扩展了文件的概念：
In Linux Everything is a File
- 目录系统提供了组织文件的结构：
目录也是一种文件

文件类型

- 普通文件 Regular Files
- 目录文件 Directory Files
- 特殊文件 Special Files
 - 设备
 - 管道
 - 套接字
 - ...

文件类型

- 普通文件 Regular Files
- 目录文件 Directory Files
- 特殊文件 Special Files
 - 设备
 - 管道
 - 套接字
 - ...

文件类型

- 普通文件 Regular Files
- 目录文件 Directory Files
- 特殊文件 Special Files
 - 设备
 - 管道
 - 套接字
 - ...

文件类型

- 普通文件 Regular Files
- 目录文件 Directory Files
- 特殊文件 Special Files
 - 设备
 - 管道
 - 套接字
 - ...

文件类型

- 普通文件 Regular Files
- 目录文件 Directory Files
- 特殊文件 Special Files
 - 设备
 - 管道
 - 套接字
 - ...

文件（目录）名

- 文件名可以由字母、数字和其他字符组成
- 某些字符有特别含义，尽量避免使用
- “/” 字符不能用于一般文件名中
- 根目录的名字只能是 “/”

文件（目录）名

- 文件名可以由字母、数字和其他字符组成
- 某些字符有特别含义，尽量避免使用
- “/” 字符不能用于一般文件名中
- 根目录的名字只能是 “/”

文件（目录）名

- 文件名可以由字母、数字和其他字符组成
- 某些字符有特殊含义，尽量避免使用
- “/” 字符不能用于一般文件名中
- 根目录的名字只能是 “/”

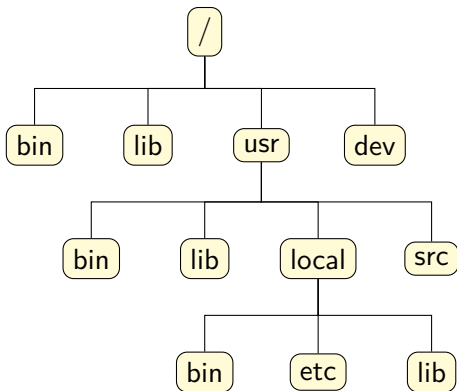
文件（目录）名

- 文件名可以由字母、数字和其他字符组成
- 某些字符有特别含义，尽量避免使用
- “/” 字符不能用于一般文件名中
- 根目录的名字只能是 “/”

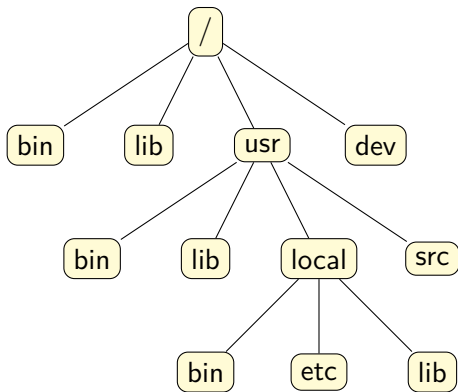
层次目录

- 根目录
- 子目录
- 父目录
- 树型结构

例子：Linux 的目录结构



例子：Linux 的目录结构



用户主目录

The Home Directory

作为一个“多用户”的操作系统，每个用户拥有各自独立的“主目录”，如：

```
/home/david,  
/home/tom,  
/home/jack,  
/home/mary
```

一般情况下，普通用户没有访问其他用户主目录的权限。

用户主目录

The Home Directory

作为一个“多用户”的操作系统，每个用户拥有各自独立的“主目录”，如：

```
/home/david,  
/home/tom,  
/home/jack,  
/home/mary
```

一般情况下，普通用户没有访问其他用户主目录的权限。

工作目录

The Working Directory

用户的很多操作，都与一个特定的目录相关联。

工作目录又称“当前目录”

可以用 `cd` 命令把工作目录调整到目录树上的任一位置。（需要对应的权限）

路径和路径名

路径用来标识文件或目录的逻辑位置。

路径由一连串的目录名组成，中间由“/”分隔。

(与 Windows 下的分隔符号的方向相反)

路径的文件或目录名，称为文件或目录的“路径名”。

- 绝对路径名
- 相对路径名

路径和路径名

路径用来标识文件或目录的逻辑位置。

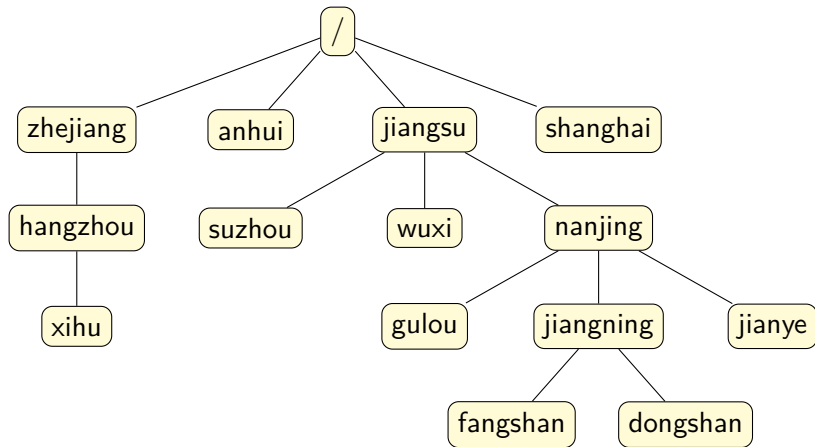
路径由一连串的目录名组成，中间由“/”分隔。

(与 Windows 下的分隔符号的方向相反)

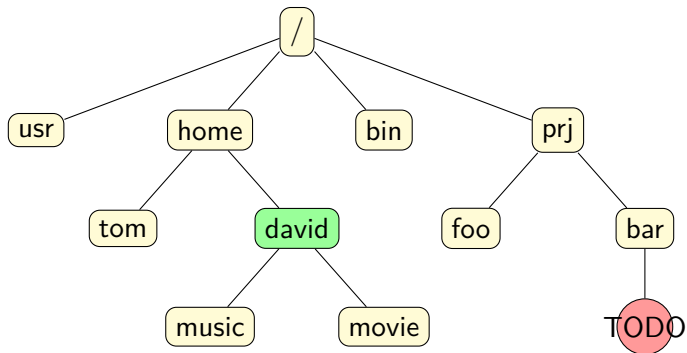
路径的文件或目录名，称为文件或目录的“路径名”。

- 绝对路径名
- 相对路径名

目录树的例子



一个假想的目录树



针对目录的操作

- 显示工作目录的目录名：pwd
- 改变工作目录：cd
- 创建目录：mkdir
- 删除目录：rmdir
- 目录列表：ls

针对目录的操作

- 显示工作目录的目录名：pwd
- 改变工作目录：cd
- 创建目录：mkdir
- 删除目录：rmdir
- 目录列表：ls

针对目录的操作

- 显示工作目录的目录名：pwd
- 改变工作目录：cd
- 创建目录：mkdir
- 删除目录：rmdir
- 目录列表：ls

针对目录的操作

- 显示工作目录的目录名：pwd
- 改变工作目录：cd
- 创建目录：mkdir
- 删除目录：rmdir
- 目录列表：ls

针对目录的操作

- 显示工作目录的目录名：pwd
- 改变工作目录：cd
- 创建目录：mkdir
- 删除目录：rmdir
- 目录列表：ls

小结

- 目录和文件的概念
- 路径
- 针对目录的操作

Part IV

Linux 文件系统基础（续）

ls 命令的各种形式

```
ls
```

```
ls /prj/foo  
ls /prj/foo/TODO  
ls ../prj/foo  
ls bar
```

```
cd /prj/foo  
ls
```

```
ls -l -a music  
ls -la music
```

ls 命令的各种形式

```
ls
```

```
ls /prj/foo  
ls /prj/foo/TODO  
ls ../prj/foo  
ls bar
```

```
cd /prj/foo  
ls
```

```
ls -l -a music  
ls -la music
```

ls 命令的各种形式

```
ls
```

```
ls /prj/foo  
ls /prj/foo/TODO  
ls ../prj/foo  
ls bar
```

```
cd /prj/foo  
ls
```

```
ls -l -a music  
ls -la music
```

ls 命令的各种形式

```
ls
```

```
ls /prj/foo  
ls /prj/foo/TODO  
ls ../prj/foo  
ls bar
```

```
cd /prj/foo  
ls
```

```
ls -l -a music  
ls -la music
```

隐藏文件

- 特殊的文件名：“.”开头的文件
- 每个目录中都有的两个特殊文件：. 和..
- 隐藏文件不是秘密
- 隐藏文件的应用场合
- 以上也适用于隐藏目录

隐藏文件

- 特殊的文件名：“.”开头的文件
- 每个目录中都有的两个特殊文件：. 和..
- 隐藏文件不是秘密
- 隐藏文件的应用场合
- 以上也适用于隐藏目录

隐藏文件

- 特殊的文件名：“.”开头的文件
- 每个目录中都有的两个特殊文件：. 和..
- 隐藏文件不是秘密
- 隐藏文件的应用场合
- 以上也适用于隐藏目录

隐藏文件

- 特殊的文件名：“.”开头的文件
- 每个目录中都有的两个特殊文件：. 和..
- 隐藏文件不是秘密
- 隐藏文件的应用场合
- 以上也适用于隐藏目录

隐藏文件

- 特殊的文件名：“.”开头的文件
- 每个目录中都有的两个特殊文件：. 和..
- 隐藏文件不是秘密
- 隐藏文件的应用场合
- 以上也适用于隐藏目录

文件的详细信息

- 文件类型
- 文件访问模式（权限）
- 链接数
- 属主
- 组
- 大小
- 上次修改日期
- 文件名

文件类型

- r - x r - x r - x

- r w - r - - - -

d r w x r w x r w x

- 普通文件
d 目录
l 符号链接
b 块设备
c 字符设备

...

文件访问模式（权限）

-	r	-	x	r	-	x	r	-	x
-	r	w	-	r	-	-	-	-	-
d	r	w	x	r	w	x	r	w	x

文件访问模式

文件访问模式（权限）

-	r	-	x	r	-	x	r	-	x
-	r	w	-	r	-	-	-	-	-
d	r	w	x	r	w	x	r	w	x

-	不可读
r	可读的

文件访问模式（权限）

-	r	-	x	r	-	x	r	-	x
-	r	w	-	r	-	-	-	-	-
d	r	w	x	r	w	x	r	w	x

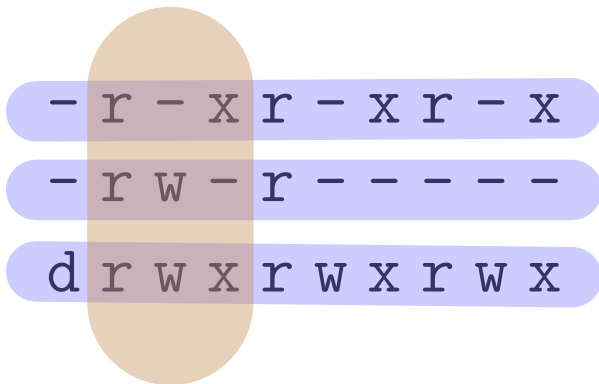
-	不可写
w	可写的

文件访问模式（权限）

```
- r - x r - x r - x  
- r w - r - - - - -  
d r w x r w x r w x
```

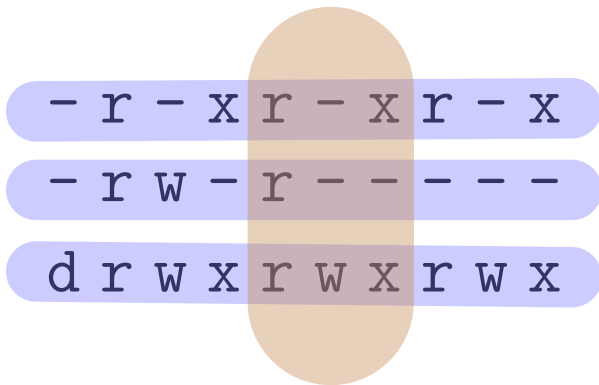
-	不可执行
x	可执行的

文件访问模式（权限）



u 文件所有者

文件访问模式（权限）



g 文件组用户

文件访问模式（权限）

- r - x r - x r - x

- r w - r - - - - -

d r w x r w x r w x

o 其他用户

改变文件或目录的访问模式

```
chmod 777 foo  
chmod 755 bar  
chmod 600 blabla
```

```
chmod g+w foo  
chmod go-x foo  
chmod u=rw,go-x foo  
chmod a=rwx foo
```

改变文件或目录的访问模式

```
chmod 777 foo  
chmod 755 bar  
chmod 600 blabla
```

```
chmod g+w foo  
chmod go-x foo  
chmod u=rw,go-x foo  
chmod a=rwx foo
```

改变文件或目录的访问模式

```
chmod 777 foo  
chmod 755 bar  
chmod 600 blabla
```

```
chmod g+w foo  
chmod go-x foo  
chmod u=rw,go-x foo  
chmod a=rwx foo
```

针对文件的其他操作

- 显示文件内容：cat
原意 concatenate，连接。
- 打印命令：lp, lpr
- 取消打印：cancel
- 打印机状态：lpstat
- 删除文件：rm
注意：一定要了解删除文件的风险！

回顾假想情境

Tom 与 David 合作完成一个项目。

Tom 写了一个便条，保存成文件，给 David 看。

Tom 的操作：

- 1 登录
- 2 新建目录
- 3 新建文件
- 4 撰写内容
- 5 退出

David 的操作：

- 1 登录
- 2 找到文件
- 3 查看文件内容
- 4 删除文件
- 5 退出

回顾假想情境

Tom 与 David 合作完成一个项目。

Tom 写了一个便条，保存成文件，给 David 看。

Tom 的操作：

- ① 登录
- ② 新建目录
- ③ 新建文件
- ④ 撰写内容
- ⑤ 退出

David 的操作：

- ① 登录
- ② 找到文件
- ③ 查看文件内容
- ④ 删除文件
- ⑤ 退出

回顾假想情境

Tom 与 David 合作完成一个项目。

Tom 写了一个便条，保存成文件，给 David 看。

Tom 的操作：

- ① 登录
- ② 新建目录
- ③ 新建文件
- ④ 撰写内容
- ⑤ 退出

David 的操作：

- ① 登录
- ② 找到文件
- ③ 查看文件内容
- ④ 删除文件
- ⑤ 退出

操作示范

- 以超级用户身份建立两个普通用户
- 模拟 Tom 的操作过程
- 模拟 David 的操作过程
- 留意未曾学习过的命令
- 总结整个操作过程涉及的概念

Part V

深入 Linux 文件系统

文件操作

- 创建文件
- 复制文件
- 移动文件位置
- 文件改名
- 删除文件

创建文件：初识 vi 编辑器

- 用 vi 编辑器新建一个文件，命令如下

```
vi <文件名>
```

- 注意编辑器的初始画面
- 按 `i` 键进入 文本模式
- 输入文本
- 按 `Esc` 键回到 命令模式
- 使用以下命令，保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

创建文件：初识 vi 编辑器

- 用 vi 编辑器新建一个文件，命令如下

```
vi <文件名>
```

- 注意编辑器的初始画面
- 按 `i` 键进入 文本模式
- 输入文本
- 按 `Esc` 键回到 命令模式
- 使用以下命令，保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

创建文件：初识 vi 编辑器

- 用 vi 编辑器新建一个文件，命令如下

```
vi <文件名>
```

- 注意编辑器的初始画面
- 按 **i** 键进入 文本模式
- 输入文本
- 按 **Esc** 键回到 命令模式
- 使用以下命令，保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

创建文件：初识 vi 编辑器

- 用 vi 编辑器新建一个文件，命令如下

```
vi <文件名>
```

- 注意编辑器的初始画面
- 按 **i** 键进入 文本模式
- 输入文本
- 按 **Esc** 键回到 命令模式
- 使用以下命令，保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

创建文件：初识 vi 编辑器

- 用 vi 编辑器新建一个文件，命令如下

```
vi <文件名>
```

- 注意编辑器的初始画面
- 按 **i** 键进入 文本模式
- 输入文本
- 按 **Esc** 键回到 命令模式
- 使用以下命令，保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

创建文件：初识 vi 编辑器

- 用 vi 编辑器新建一个文件，命令如下

```
vi <文件名>
```

- 注意编辑器的初始画面
- 按 **i** 键进入 文本模式
- 输入文本
- 按 **Esc** 键回到 命令模式
- 使用以下命令，保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

复制文件

- cp 命令的一般形式

```
cp originalfile copiedfile
```

- cp 命令的风险

如已经存在同名的目标文件，将会被覆盖！

使用 cp 命令的选项，可以降低风险。另外的选项，增加 cp 命令的灵活性。

–b 备份目标文件

–i 覆盖确认提示

–r 连同目录复制

- 问题：修改其中一个文件的内容，是否影响另外一个复本的内容？

复制文件

- cp 命令的一般形式

```
cp originalfile copiedfile
```

- cp 命令的风险

如已经存在同名的目标文件，将会被覆盖！

使用 cp 命令的选项，可以降低风险。另外的选项，增加 cp 命令的灵活性。

—b 备份目标文件

—i 覆盖确认提示

—r 连同目录复制

- 问题：修改其中一个文件的内容，是否影响另外一个复本的内容？

复制文件

- cp 命令的一般形式

```
cp originalfile copiedfile
```

- cp 命令的风险

如已经存在同名的目标文件，将会被覆盖！

使用 cp 命令的选项，可以降低风险。另外的选项，增加 cp 命令的灵活性。

—b 备份目标文件

—i 覆盖确认提示

—r 连同目录复制

- 问题：修改其中一个文件的内容，是否影响另外一个复本的内容？

复制文件

- cp 命令的一般形式

```
cp originalfile copiedfile
```

- cp 命令的风险

如已经存在同名的目标文件，将会被覆盖！

使用 cp 命令的选项，可以降低风险。另外的选项，增加 cp 命令的灵活性。

- b 备份目标文件

- i 覆盖确认提示

- r 连同目录复制

- 问题：修改其中一个文件的内容，是否影响另外一个复本的内容？

复制文件

- cp 命令的一般形式

```
cp originalfile copiedfile
```

- cp 命令的风险

如已经存在同名的目标文件，将会被覆盖！

使用 cp 命令的选项，可以降低风险。另外的选项，增加 cp 命令的灵活性。

- b 备份目标文件

- i 覆盖确认提示

- r 连同目录复制

- 问题：修改其中一个文件的内容，是否影响另外一个复本的内容？

复制文件

- cp 命令的一般形式

```
cp originalfile copiedfile
```

- cp 命令的风险

如已经存在同名的目标文件，将会被覆盖！

使用 cp 命令的选项，可以降低风险。另外的选项，增加 cp 命令的灵活性。

- b 备份目标文件
- i 覆盖确认提示
- r 连同目录复制

- 问题：修改其中一个文件的内容，是否影响另外一个复本的内容？

复制文件

- cp 命令的一般形式

```
cp originalfile copiedfile
```

- cp 命令的风险

如已经存在同名的目标文件，将会被覆盖！

使用 cp 命令的选项，可以降低风险。另外的选项，增加 cp 命令的灵活性。

- b 备份目标文件
- i 覆盖确认提示
- r 连同目录复制

- 问题：修改其中一个文件的内容，是否影响另外一个复本的内容？

移动文件

- mv 命令的一般形式

```
mv /path/to/old_dir/filename /path/to/new_dir
```

- mv 命令的风险和预防
- mv 命令的实际动作和复制不同
- mv 命令用于目录时，整个子目录连同下级目录一起移动

移动文件

- mv 命令的一般形式

```
mv /path/to/old_dir/filename /path/to/new_dir
```

- mv 命令的风险和预防

- mv 命令的实际动作和复制不同

- mv 命令用于目录时，整个子目录连同下级目录一起移动

移动文件

- mv 命令的一般形式

```
mv /path/to/old_dir/filename /path/to/new_dir
```

- mv 命令的风险和预防
- mv 命令的实际动作和复制不同
- mv 命令用于目录时，整个子目录连同下级目录一起移动

移动文件

- mv 命令的一般形式

```
mv /path/to/old_dir/filename /path/to/new_dir
```

- mv 命令的风险和预防
- mv 命令的实际动作和复制不同
- mv 命令用于目录时，整个子目录连同下级目录一起移动

文件改名

- 文件改名（重命名）仍然使用 mv 命令

```
mv oldname newname
```

- 移动文件的位置，和对文件重新命名本质相同
- mv 并不复制文件内容，只修改指向文件的索引
- 猜想：对于比较大的文件，执行 cp 和 mv 命令的时间，哪个更长？

文件改名

- 文件改名（重命名）仍然使用 mv 命令

```
mv oldname newname
```

- 移动文件的位置，和对文件重新命名本质相同
- mv 并不复制文件内容，只修改指向文件的索引
- 猜想：对于比较大的文件，执行 cp 和 mv 命令的时间，哪个更长？

文件改名

- 文件改名（重命名）仍然使用 mv 命令

```
mv oldname newname
```

- 移动文件的位置，和对文件重新命名本质相同
- mv 并不复制文件内容，只修改指向文件的索引
- 猜想：对于比较大的文件，执行 cp 和 mv 命令的时间，哪个更长？

文件改名

- 文件改名（重命名）仍然使用 mv 命令

```
mv oldname newname
```

- 移动文件的位置，和对文件重新命名本质相同
- mv 并不复制文件内容，只修改指向文件的索引
- 猜想：对于比较大的文件，执行 cp 和 mv 命令的时间，哪个更长？

删除文件

- rm 命令的一般形式

```
rm somefile
```

- 删除命令的风险

警告：没有所谓“恢复删除”的命令。所以使用删除命令要仔细！

使用 rm -i 选项，可以减少发生灾难的机会。

```
rm -i somefile
```

删除文件

- rm 命令的一般形式

```
rm somefile
```

- 删除命令的风险

警告：没有所谓“恢复删除”的命令。所以使用删除命令要仔细！

使用 rm -i 选项，可以减少发生灾难的机会。

```
rm -i somefile
```

删除文件

- rm 命令的一般形式

```
rm somefile
```

- 删除命令的风险

警告：没有所谓“恢复删除”的命令。所以使用删除命令要仔细！

使用 rm -i 选项，可以减少发生灾难的机会。

```
rm -i somefile
```

删除目录

- 回顾：`rmdir` 命令只能删除空目录
- 一般的 `rm` 命令不能删除目录
- `rm` 加上 `-r` 选项，可以删除目录（空或非空）

```
rm -r somedir
```

- 术语：递归删除
- 一个强大而危险的命令：

```
rm -rf /some/important/dir
```

- 这个最危险的命令，也是被使用最为频繁的命令。

删除目录

- 回顾：rmdir 命令只能删除空目录
- 一般的 rm 命令不能删除目录

- rm 加上 -r 选项，可以删除目录（空或非空）

```
rm -r somedir
```

- 术语：递归删除
- 一个强大而危险的命令：

```
rm -rf /some/important/dir
```

- 这个最危险的命令，也是被使用最为频繁的命令。

删除目录

- 回顾：rmdir 命令只能删除空目录
- 一般的 rm 命令不能删除目录
- rm 加上 -r 选项，可以删除目录（空或非空）

```
rm -r somedir
```

- 术语：递归删除
- 一个强大而危险的命令：

```
rm -rf /some/important/dir
```

- 这个最危险的命令，也是被使用最为频繁的命令。

删除目录

- 回顾：rmdir 命令只能删除空目录
- 一般的 rm 命令不能删除目录
- rm 加上 -r 选项，可以删除目录（空或非空）

```
rm -r somedir
```

- 术语：递归删除
- 一个强大而危险的命令：

```
rm -rf /some/important/dir
```

- 这个最危险的命令，也是被使用最为频繁的命令。

删除目录

- 回顾：rmdir 命令只能删除空目录
- 一般的 rm 命令不能删除目录
- rm 加上 -r 选项，可以删除目录（空或非空）

```
rm -r somedir
```

- 术语：递归删除
- 一个强大而危险的命令：

```
rm -rf /some/important/dir
```

- 这个最危险的命令，也是被使用最为频繁的命令。

删除目录

- 回顾：rmkdir 命令只能删除空目录
- 一般的 rm 命令不能删除目录
- rm 加上 -r 选项，可以删除目录（空或非空）

```
rm -r somedir
```

- 术语：递归删除
- 一个强大而危险的命令：

```
rm -rf /some/important/dir
```

- 这个最危险的命令，也是被使用最为频繁的命令。

链接

- 链接是什么？
指向文件的一个名字。

- 链接有两类：

- 链接（硬链接）
- 符号链接

- 创建链接

```
ln existed_file new_name
```

- 创建符号链接

```
ln -s existed_file another_name
```

链接

- 链接是什么？
指向文件的一个名字。
- 链接有两类：
 - 链接（硬链接）
 - 符号链接
- 创建链接

```
ln existed_file new_name
```

- 创建符号链接

```
ln -s existed_file another_name
```

链接

- 链接是什么？
指向文件的一个名字。
- 链接有两类：
 - 链接（硬链接）
 - 符号链接
- 创建链接

```
ln existed_file new_name
```

- 创建符号链接

```
ln -s existed_file another_name
```

链接

- 链接是什么？
指向文件的一个名字。
- 链接有两类：
 - 链接（硬链接）
 - 符号链接
- 创建链接

```
ln existed_file new_name
```

- 创建符号链接

```
ln -s existed_file another_name
```

链接

- 链接是什么？
指向文件的一个名字。
- 链接有两类：
 - 链接（硬链接）
 - 符号链接
- 创建链接

```
ln existed_file new_name
```

- 创建符号链接

```
ln -s existed_file another_name
```

链接

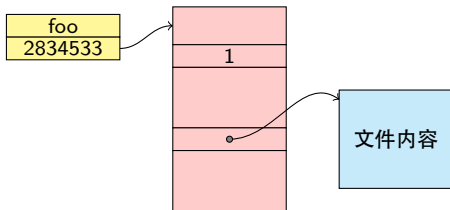
- 链接是什么？
指向文件的一个名字。
- 链接有两类：
 - 链接（硬链接）
 - 符号链接
- 创建链接

```
ln existed_file new_name
```

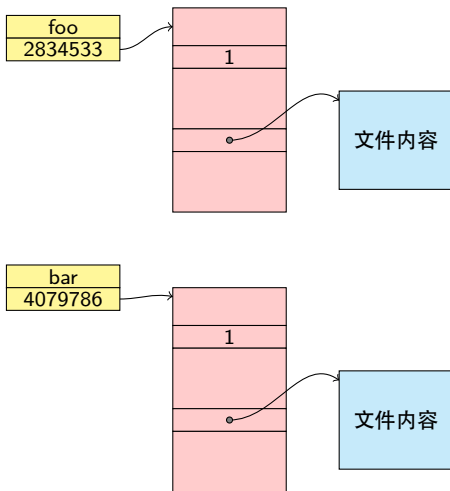
- 创建符号链接

```
ln -s existed_file another_name
```

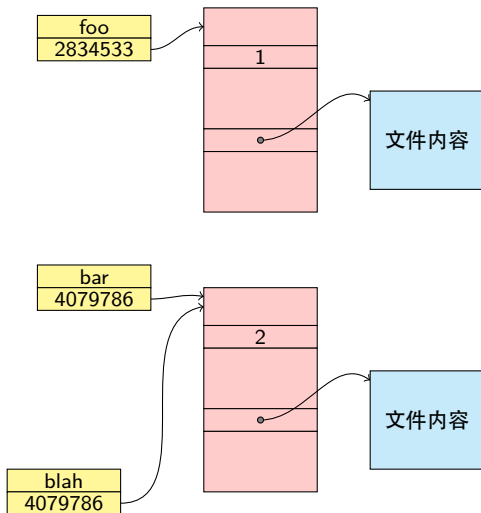

链接与符号链接



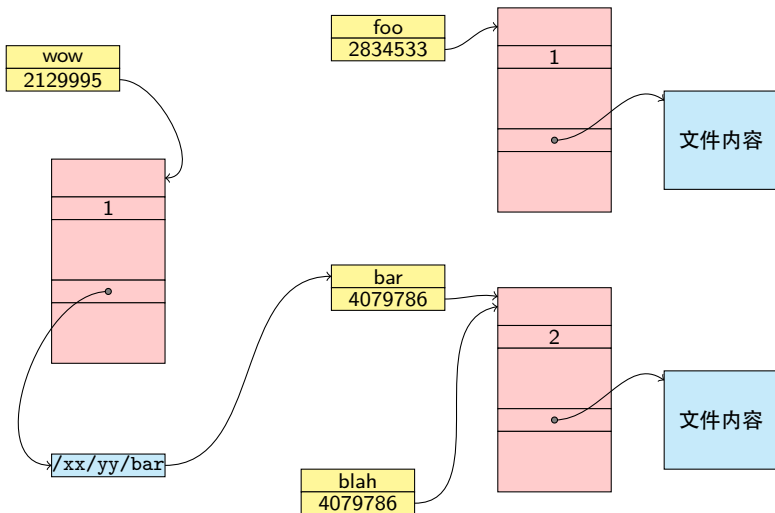
链接与符号链接



链接与符号链接



链接与符号链接



链接操作演示

- 建立链接的操作
- 更改其中一个文件的内容，是否影响链接对应的文件？
- 文件信息中链接数的变化
- i-node 信息
- 符号链接与硬链接的区别

Part VI

文件内容操作

查看文件内容

- 使用文本编辑器查看
- 使用 cat 命令查看
- 几种分页查看命令
pg, more, less 命令
- 查看文件的局部内容
head, tail, cut 命令

查看文件内容

- 使用文本编辑器查看
- 使用 cat 命令查看
- 几种分页查看命令
pg, more, less 命令
- 查看文件的局部内容
head, tail, cut 命令

查看文件内容

- 使用文本编辑器查看
- 使用 cat 命令查看
- 几种分页查看命令
pg, more, less 命令
- 查看文件的局部内容
head, tail, cut 命令

查看文件内容

- 使用文本编辑器查看
- 使用 cat 命令查看
- 几种分页查看命令
pg, more, less 命令
- 查看文件的局部内容
head, tail, cut 命令

使用 vi 编辑器查看文件

- 以只读方式启动 vi 编辑器

```
vi -R filename
```

- 等效的命令

```
view filename
```

- 使用 vi 编辑器的翻页命令前后查看
- 退出

```
:q 或者 :q! 或者 ZQ
```

使用 vi 编辑器查看文件

- 以只读方式启动 vi 编辑器

```
vi -R filename
```

- 等效的命令

```
view filename
```

- 使用 vi 编辑器的翻页命令前后查看
- 退出

```
:q 或者 :q! 或者 ZQ
```

使用 vi 编辑器查看文件

- 以只读方式启动 vi 编辑器

```
vi -R filename
```

- 等效的命令

```
view filename
```

- 使用 vi 编辑器的翻页命令前后查看

- 退出

```
:q 或者 :q! 或者 ZQ
```

使用 vi 编辑器查看文件

- 以只读方式启动 vi 编辑器

```
vi -R filename
```

- 等效的命令

```
view filename
```

- 使用 vi 编辑器的翻页命令前后查看
- 退出

```
:q 或者 :q! 或者 ZQ
```

cat 命令

cat 将文件内容发送至标准输出

```
cat file
```

长文件的滚屏控制

Ctrl-S	暂停
Ctrl-Q	恢复

cat 查看多个文件内容

```
cat fileA fileB fileC
```

cat 命令

cat 将文件内容发送至标准输出

```
cat file
```

长文件的滚屏控制

Ctrl-S	暂停
Ctrl-Q	恢复

cat 查看多个文件内容

```
cat fileA fileB fileC
```


cat 命令

cat 将文件内容发送至标准输出

```
cat file
```

长文件的滚屏控制

Ctrl-S	暂停
Ctrl-Q	恢复

cat 查看多个文件内容

```
cat fileA fileB fileC
```

分页查看工具

pg

```
pg longfile
```

more

```
more longfile
```

less

```
less longfile
```

h 帮助

q 退出

回车 下一行

空格 下一屏

b 回滚一屏

分页查看工具

pg

```
pg longfile
```

more

```
more longfile
```

less

```
less longfile
```

h 帮助

q 退出

回车 下一行

空格 下一屏

b 回滚一屏

分页查看工具

pg

```
pg longfile
```

more

```
more longfile
```

less

```
less longfile
```

h 帮助

q 退出

回车 下一行

空格 下一屏

b 回滚一屏

分页查看工具

pg

```
pg longfile
```

more

```
more longfile
```

less

```
less longfile
```

h 帮助

q 退出

回车 下一行

空格 下一屏

b 回滚一屏

查看文件局部内容

- 查看头部
- 查看尾部
- 查看中部
- 查看特定的列

head 命令

- 缺省参数，显示文件的头 10 行

```
head longfile
```

- 显示文件的前 26 行

```
head -n 26 longfile
```

- 从头开始显示文件，除了尾部最后的 9 行

```
head -n -9 longfile
```

head 命令

- 缺省参数，显示文件的头 10 行

```
head longfile
```

- 显示文件的前 26 行

```
head -n 26 longfile
```

- 从头开始显示文件，除了尾部最后的 9 行

```
head -n -9 longfile
```


head 命令

- 缺省参数，显示文件的头 10 行

```
head longfile
```

- 显示文件的前 26 行

```
head -n 26 longfile
```

- 从头开始显示文件，除了尾部最后的 9 行

```
head -n -9 longfile
```

tail 命令

- 缺省参数，显示文件的末尾 10 行

```
tail longfile
```

- 显示文件的末尾 26 行

```
tail -n 26 longfile
```

- 跳过前部，从第 9 行开始显示文件

```
tail -n +9 longfile
```

tail 命令

- 缺省参数，显示文件的末尾 10 行

```
tail longfile
```

- 显示文件的末尾 26 行

```
tail -n 26 longfile
```

- 跳过前部，从第 9 行开始显示文件

```
tail -n +9 longfile
```

tail 命令

- 缺省参数，显示文件的末尾 10 行

```
tail longfile
```

- 显示文件的末尾 26 行

```
tail -n 26 longfile
```

- 跳过前部，从第 9 行开始显示文件

```
tail -n +9 longfile
```

显示文件中间的部分

- 没有一个专用的命令
- 联合使用 head 和 tail 命令
- 显示文件的第 24 行到第 26 行

```
head -n 26 longfile | tail -n 3
```

- 另一种方法

```
tail -n +24 longfile | head -n 3
```

显示文件中间的部分

- 没有一个专用的命令
- 联合使用 head 和 tail 命令
- 显示文件的第 24 行到第 26 行

```
head -n 26 longfile | tail -n 3
```

- 另一种方法

```
tail -n +24 longfile | head -n 3
```

显示文件中间的部分

- 没有一个专用的命令
- 联合使用 head 和 tail 命令
- 显示文件的第 24 行到第 26 行

```
head -n 26 longfile | tail -n 3
```

- 另一种方法

```
tail -n +24 longfile | head -n 3
```

显示文件中间的部分

- 没有一个专用的命令
- 联合使用 head 和 tail 命令
- 显示文件的第 24 行到第 26 行

```
head -n 26 longfile | tail -n 3
```

- 另一种方法

```
tail -n +24 longfile | head -n 3
```


- Linux 的哲学思想之一：

Small tools cooperate with standardized inputs and outputs.

管道：Linux 系统中，一种进程通信的重要机制！

- Linux 的哲学思想之一：

Small tools cooperate with standardized inputs and outputs.

管道：Linux 系统中，一种进程通信的重要机制！

查看文件的某些列

cut 分离出文件中指定的列

```
cut -f1,7 -d: /etc/passwd  
cut -f2-5 /path/to/file
```

合并文件内容

- 多个文件连结到一起
- 文件按行合并
- 输出重定向

cat 命令

- 用 cat 命令连结多个文件
concatenate：连结

- 例子

```
cat fileA fileB fileC > file_all_in_one
```

- 符号 ">" 表示输出重定向
- 符号 ">>" 表示输出重定向，追加方式

```
cat fileA > file_all_in_one  
cat fileB >> file_all_in_one  
cat fileC >> file_all_in_one
```

cat 命令

- 用 cat 命令连结多个文件
concatenate：连结
- 例子

```
cat fileA fileB fileC > file_all_in_one
```

- 符号 “>” 表示输出重定向
- 符号 “>>” 表示输出重定向，追加方式

```
cat fileA > file_all_in_one  
cat fileB >> file_all_in_one  
cat fileC >> file_all_in_one
```

cat 命令

- 用 cat 命令连结多个文件

concatenate：连结

- 例子

```
cat fileA fileB fileC > file_all_in_one
```

- 符号 “>” 表示输出重定向
- 符号 “>>” 表示输出重定向，追加方式

```
cat fileA > file_all_in_one  
cat fileB >> file_all_in_one  
cat fileC >> file_all_in_one
```

cat 命令

- 用 cat 命令连结多个文件

concatenate：连结

- 例子

```
cat fileA fileB fileC > file_all_in_one
```

- 符号 “>” 表示输出重定向
- 符号 “>>” 表示输出重定向，追加方式

```
cat fileA > file_all_in_one  
cat fileB >> file_all_in_one  
cat fileC >> file_all_in_one
```


按行合并文件

paste 命令

```
paste one two
```

将两个文件按行合并后显示。

paste 命令输出到文件

```
paste one two > one_and_two_side_by_side
```

将两个文件按行合并生成新的文件。

复制文件的不同方法

方法一

```
cp fileA fileB
```

方法二

```
cat fileA > fileB
```

文件改名的不同方法

方法一

```
mv fileA fileB
```

方法二

```
cp fileA fileB  
rm fileA
```

方法三

```
ln fileA fileB  
rm fileA
```

综合举例

合并两个文件

tel-a:

no	ct	pre	fee
1	nj	025	10
2	gz	020	30
3	sh	021	70

tel-b:

no	ct	pre	fee
4	bj	010	40
5	wh	027	60
6	cd	028	60

合并成为：

telbook:

no	ct	pre
1	nj	025
2	gz	020
3	sh	021
4	bj	010
5	wh	027
6	cd	028

Part VII

vi 编辑器初步

回顾：用 vi 创建新文件

- 用 vi 编辑器新建一个文件，命令如下

```
vi <文件名>
```

- 按 **i** 键进入 文本模式
- 输入文本
- 按 **Esc** 键回到 命令模式
- 使用以下命令，保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

回顾：用 vi 查看文件

- 以只读方式启动 vi 编辑器

```
vi -R filename
```

- 使用 vi 编辑器的翻页命令前后查看

即将学习

- 退出

```
:q    或者    :q!   或者    ZQ
```


文本编辑器

编辑器是创建修改文本文件的基本工具。
编辑器所处理的目标，是纯文本文件（Plain Text File）。

两种编辑器的基本类型：

- 行编辑器（ed, ex）
- 全屏编辑器（vi, emacs）

注意区分普通编辑器，具有排版与格式控制功能的字处理器。

文本编辑器

编辑器是创建修改文本文件的基本工具。
编辑器所处理的目标，是纯文本文件（Plain Text File）。

两种编辑器的基本类型：

- 行编辑器（ed, ex）
- 全屏编辑器（vi, emacs）

注意区分普通编辑器，具有排版与格式控制功能的字处理器。

文本编辑器

编辑器是创建修改文本文件的基本工具。
编辑器所处理的目标，是纯文本文件（Plain Text File）。

两种编辑器的基本类型：

- 行编辑器（ed, ex）
- 全屏编辑器（vi, emacs）

注意区分普通编辑器，具有排版与格式控制功能的字处理器。

Linux 系统上的各种编辑器

- ed
- ex
- vi
- emacs
- pico
- nano
- gedit
- kate
- kwrite
- leafpad

vi 编辑器

- 每一种 Linux 系统的缺省配置
- 支持几乎所有的操作系统 (Linux/Mac/Windows/...)
- 可以工作在字符方式下
- 对初学者来说, 有点 “不自然”
- 高效、强大
- 有多种变种程序
- vim 是一种改进的 vi

vi 编辑器

- 每一种 Linux 系统的缺省配置
- 支持几乎所有的操作系统 (Linux/Mac/Windows/...)
- 可以工作在字符方式下
- 对初学者来说, 有点 “不自然”
- 高效、强大
- 有多种变种程序
- vim 是一种改进的 vi

vi 编辑器

- 每一种 Linux 系统的缺省配置
- 支持几乎所有的操作系统 (Linux/Mac/Windows/...)
- 可以工作在字符方式下
- 对初学者来说, 有点 “不自然”
- 高效、强大
- 有多种变种程序
- vim 是一种改进的 vi

vi 编辑器

- 每一种 Linux 系统的缺省配置
- 支持几乎所有的操作系统 (Linux/Mac/Windows/...)
- 可以工作在字符方式下
- 对初学者来说, 有点 “不自然”
- 高效、强大
- 有多种变种程序
- vim 是一种改进的 vi

vi 编辑器

- 每一种 Linux 系统的缺省配置
- 支持几乎所有的操作系统 (Linux/Mac/Windows/...)
- 可以工作在字符方式下
- 对初学者来说, 有点 “不自然”
- 高效、强大
- 有多种变种程序
- vim 是一种改进的 vi

vi 编辑器

- 每一种 Linux 系统的缺省配置
- 支持几乎所有的操作系统 (Linux/Mac/Windows/...)
- 可以工作在字符方式下
- 对初学者来说, 有点 “不自然”
- 高效、强大
- 有多种变种程序
- vim 是一种改进的 vi

vi 编辑器

- 每一种 Linux 系统的缺省配置
- 支持几乎所有的操作系统 (Linux/Mac/Windows/...)
- 可以工作在字符方式下
- 对初学者来说, 有点 “不自然”
- 高效、强大
- 有多种变种程序
- vim 是一种改进的 vi

vi 的启动与退出

- 启动 vi, 命令如下

```
vi <文件名>
```

- 各种编辑操作
- 保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

vi 的启动与退出

- 启动 vi, 命令如下

```
vi <文件名>
```

- 各种编辑操作
- 保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

vi 的启动与退出

- 启动 vi, 命令如下

```
vi <文件名>
```

- 各种编辑操作
- 保存文件并退出 vi 编辑器

```
:wq <回车>
```

或者 ZZ

关于保存和退出的变化

- 保存文件（不退出，继续编辑）

```
:w <回车>
```

- 退出编辑器（若已保存，将退出；若有改动未保存，将拒绝退出）

```
:q <回车>
```

- 保存文件并退出 vi 编辑器

```
:wq <回车>
```

- 有改动，但不想保存，放弃所做的修改，坚决退出

```
:q! <回车>
```

编辑发生在缓冲区中，未经保存，文件内容不会变化

关于保存和退出的变化

- 保存文件（不退出，继续编辑）

```
:w <回车>
```

- 退出编辑器（若已保存，将退出；若有改动未保存，将拒绝退出）

```
:q <回车>
```

- 保存文件并退出 vi 编辑器

```
:wq <回车>
```

- 有改动，但不想保存，放弃所做的修改，坚决退出

```
:q! <回车>
```

编辑发生在缓冲区中，未经保存，文件内容不会变化

关于保存和退出的变化

- 保存文件（不退出，继续编辑）

```
:w <回车>
```

- 退出编辑器（若已保存，将退出；若有改动未保存，将拒绝退出）

```
:q <回车>
```

- 保存文件并退出 vi 编辑器

```
:wq <回车>
```

- 有改动，但不想保存，放弃所做的修改，坚决退出

```
:q! <回车>
```

编辑发生在缓冲区中，未经保存，文件内容不会变化

关于保存和退出的变化

- 保存文件（不退出，继续编辑）

```
:w <回车>
```

- 退出编辑器（若已保存，将退出；若有改动未保存，将拒绝退出）

```
:q <回车>
```

- 保存文件并退出 vi 编辑器

```
:wq <回车>
```

- 有改动，但不想保存，放弃所做的修改，坚决退出

```
:q! <回车>
```

编辑发生在缓冲区中，未经保存，文件内容不会变化

关于保存和退出的变化

- 保存文件（不退出，继续编辑）

```
:w <回车>
```

- 退出编辑器（若已保存，将退出；若有改动未保存，将拒绝退出）

```
:q <回车>
```

- 保存文件并退出 vi 编辑器

```
:wq <回车>
```

- 有改动，但不想保存，放弃所做的修改，坚决退出

```
:q! <回车>
```

编辑发生在缓冲区中，未经保存，文件内容不会变化

关于启动 vi 的变化

- 通常启动 vi 的命令如下

```
vi <文件名>
```

- 启动 vi 时不带文件名参数之后...
- 在 vi 内打开原有文件

```
:e <文件名>
```

- 或者在保存文件时，指定文件名

```
:w <文件名>
```

```
:w! <文件名>
```

强制覆盖已有的同名文件

- 也可以

```
:wq <文件名>
```

```
:wq! <文件名>
```

关于启动 vi 的变化

- 通常启动 vi 的命令如下

```
vi <文件名>
```

- 启动 vi 时不带文件名参数之后...
- 在 vi 内打开原有文件

```
:e <文件名>
```

- 或者在保存文件时，指定文件名

```
:w <文件名>
```

```
:w! <文件名>
```

强制覆盖已有的同名文件

- 也可以

```
:wq <文件名>
```

```
:wq! <文件名>
```

关于启动 vi 的变化

- 通常启动 vi 的命令如下

```
vi <文件名>
```

- 启动 vi 时不带文件名参数之后...
- 在 vi 内打开原有文件

```
:e <文件名>
```

- 或者在保存文件时，指定文件名

```
:w <文件名>
```

```
:w! <文件名>
```

强制覆盖已有的同名文件

- 也可以

```
:wq <文件名>
```

```
:wq! <文件名>
```

关于启动 vi 的变化

- 通常启动 vi 的命令如下

```
vi <文件名>
```

- 启动 vi 时不带文件名参数之后...
- 在 vi 内打开原有文件

```
:e <文件名>
```

- 或者在保存文件时，指定文件名

```
:w <文件名>
```

```
:w! <文件名>
```

强制覆盖已有的同名文件

- 也可以

```
:wq <文件名>
```

```
:wq! <文件名>
```

关于启动 vi 的变化

- 通常启动 vi 的命令如下

```
vi <文件名>
```

- 启动 vi 时不带文件名参数之后...
- 在 vi 内打开原有文件

```
:e <文件名>
```

- 或者在保存文件时，指定文件名

```
:w <文件名>
```

```
:w! <文件名>
```

强制覆盖已有的同名文件

- 也可以

```
:wq <文件名>
```

```
:wq! <文件名>
```


常用的编辑操作

功能	命令
光标移动	h, j, k, l
增	i, I, a, A, o, O
删	x, X
改	r, R

vi 模式及切换

vi 编辑器是一种“有模式”的编辑器。

vi 的两种基本工作模式：

- 刚启动的时候，vi 自动处于“命令模式”。所有的按键均被理解成命令。
- i（或者 l a A R o O ...）命令，可以令 vi 进入“文本模式”。
- Esc 键可以令 vi 由“文本模式”回到“命令模式”。
- 如果记不清当前模式，可以按两次 Esc 键，确保 vi 回到命令模式。

vi 模式及切换

vi 编辑器是一种“有模式”的编辑器。

vi 的两种基本工作模式：

- 刚启动的时候，vi 自动处于“命令模式”。所有的按键均被理解成命令。
- i（或者 I a A R o O ...）命令，可以令 vi 进入“文本模式”。
- Esc 键可以令 vi 由“文本模式”回到“命令模式”。
- 如果记不清当前模式，可以按两次 Esc 键，确保 vi 回到命令模式。

vi 模式及切换

vi 编辑器是一种“有模式”的编辑器。

vi 的两种基本工作模式：

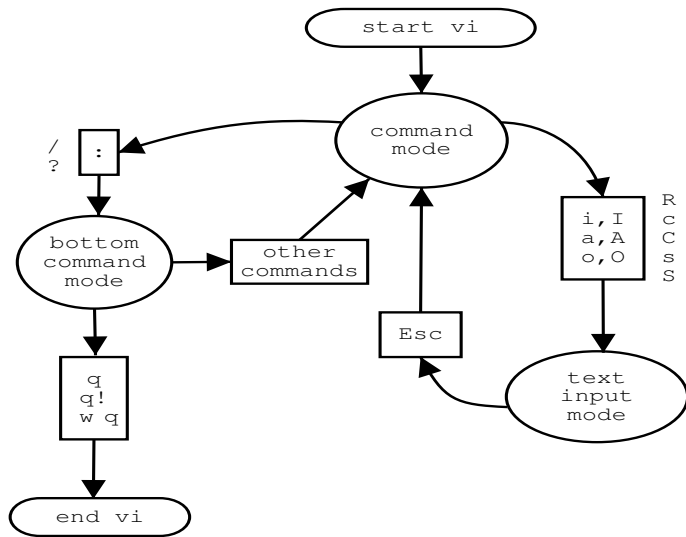
- 刚启动的时候，vi 自动处于“命令模式”。所有的按键均被理解成命令。
- i（或者 I a A R o O ...）命令，可以令 vi 进入“文本模式”。
- Esc 键可以令 vi 由“文本模式”回到“命令模式”。
- 如果记不清当前模式，可以按两次 Esc 键，确保 vi 回到命令模式。

vi 模式及切换

vi 编辑器是一种“有模式”的编辑器。

vi 的两种基本工作模式：

- 刚启动的时候，vi 自动处于“命令模式”。所有的按键均被理解成命令。
- i（或者 I a A R o O ...）命令，可以令 vi 进入“文本模式”。
- Esc 键可以令 vi 由“文本模式”回到“命令模式”。
- 如果记不清当前模式，可以按两次 Esc 键，确保 vi 回到命令模式。



按键/操作	作用
Esc	回到命令模式
i, I, a, A, o, O, ...	进入文本模式
另外一些命令	不影响模式

移动光标的更多手段

- 按单词移动光标: w, b, e, ge
- 移动到行首与行末: 0 (数字零), ^, \$
- 移动到上一段、下一段: {, }
- 本行内移动到特定字符: fx, tx, Fx, Tx (x 代表特定字符)
分号和逗号, 用来重复这样的移动操作

移动光标的更多手段

- 按单词移动光标: w, b, e, ge
- 移动到行首与行末: 0 (数字零), ^, \$
- 移动到上一段、下一段: {, }
- 本行内移动到特定字符: fx, tx, Fx, Tx (x 代表特定字符)
分号和逗号, 用来重复这样的移动操作

移动光标的更多手段

- 按单词移动光标: w, b, e, ge
- 移动到行首与行末: 0 (数字零), ^, \$
- 移动到上一段、下一段: {, }
- 本行内移动到特定字符: fx, tx, Fx, Tx (x 代表特定字符)
分号和逗号, 用来重复这样的移动操作

移动光标的更多手段

- 按单词移动光标: w, b, e, ge
- 移动到行首与行末: 0 (数字零), ^, \$
- 移动到上一段、下一段: {, }
- 本行内移动到特定字符: fx, tx, Fx, Tx (x 代表特定字符)
分号和逗号, 用来重复这样的移动操作

编辑动作命令

编辑命令	功能
d	删除
y	复制 (yank)
(小写) p	粘贴 (到光标右/下)
(大写) P	粘贴 (到光标左/上)
c	更改
s	替换

编辑范围指示

\$	到行尾
0 (数字零)	到行首
e	到词尾
w	到下一词首 (包含词后的空)
b	(退) 到词首
其他移动光标的命令	表示从当前位置到目标位置范围

动作 + 范围

- d\$ 删除到行尾
- d0 删除到行首
- ce 改变到词尾（删除到词尾，并进入文本输入模式）
- yw 复制到下一词首（包含本词后的空）
- yb 向后退方向复制到词首

动作字符重复

- dd 删除一行
- yy 复制一行
- cc 改变一行

不指定域，而将动作命令字符重复一次，是代表对当前整行的操作。

数字 + 动作 + 范围

- 3dd 删除三行
- 5yw 复制五个词
- 4p 把临时寄存器中的内容在当前位置后，复制 4 次
- 12P 把临时寄存器中的内容在当前位置前，复制 12 次

数字用于光标移动命令之前

- 23l 向右移动 23 个字符
- 5j 向下移动 5 行
- 7w 向右移动 7 个单词
- 4fe 本行之内，向右移到第 4 个字母 e 上

数字用于光标移动命令之前

- 23l 向右移动 23 个字符
- 5j 向下移动 5 行
- 7w 向右移动 7 个单词
- 4fe 本行之内，向右移到第 4 个字母 e 上

数字用于光标移动命令之前

- 23l 向右移动 23 个字符
- 5j 向下移动 5 行
- 7w 向右移动 7 个单词
- 4fe 本行之内，向右移到第 4 个字母 e 上

数字用于光标移动命令之前

- 23l 向右移动 23 个字符
- 5j 向下移动 5 行
- 7w 向右移动 7 个单词
- 4fe 本行之内，向右移到第 4 个字母 e 上

动作 + 数字 + 范围

- c3w 更改 3 个单词
- d2d 删除两行（本行及下方一行）
- y2w 复制 2 个单词
- 5p 临时寄存器的内容在右方或本行下方复制 2 次
- 3d2d 删除 6 行

动作 + 数字 + 范围

- c3w 更改 3 个单词
- d2d 删除两行（本行及下方一行）
- y2w 复制 2 个单词
- 5p 临时寄存器的内容在右方或本行下方复制 2 次
- 3d2d 删除 6 行

动作 + 数字 + 范围

- c3w 更改 3 个单词
- d2d 删除两行（本行及下方一行）
- y2w 复制 2 个单词
- 5p 临时寄存器的内容在右方或本行下方复制 2 次
- 3d2d 删除 6 行

动作 + 数字 + 范围

- c3w 更改 3 个单词
- d2d 删除两行（本行及下方一行）
- y2w 复制 2 个单词
- 5p 临时寄存器的内容在右方或本行下方复制 2 次
- 3d2d 删除 6 行

动作 + 数字 + 范围

- c3w 更改 3 个单词
- d2d 删除两行（本行及下方一行）
- y2w 复制 2 个单词
- 5p 临时寄存器的内容在右方或本行下方复制 2 次
- 3d2d 删除 6 行

搜索与重复操作

- 搜索字符串

<code>/string</code>	向前搜索
<code>?string</code>	向后搜索
<code>*</code>	向前搜索光标处的单词
<code>#</code>	向后搜索光标处的单词
<code>n</code>	下一匹配处
<code>N</code>	上一匹配处

- 重复上一次的操作

· (句点)

(技巧：搜索操作与重复操作相结合的应用，可以提高编辑效果)

搜索与重复操作

- 搜索字符串

/string	向前搜索
?string	向后搜索
*	向前搜索光标处的单词
#	向后搜索光标处的单词
n	下一匹配处
N	上一匹配处

- 重复上一次的操作

. (句点)

(技巧：搜索操作与重复操作相结合的应用，可以提高编辑效果)

搜索与重复操作

- 搜索字符串

<code>/string</code>	向前搜索
<code>?string</code>	向后搜索
<code>*</code>	向前搜索光标处的单词
<code>#</code>	向后搜索光标处的单词
<code>n</code>	下一匹配处
<code>N</code>	上一匹配处

- 重复上一次的操作

. (句点)

(技巧：搜索操作与重复操作相结合的应用，可以提高编辑效果)

undo 机制

u	取消上一次的改动（回退一步）
Ctrl-R	撤销一次回退操作
U	一次性取消最近在本行上的全部改动

在线帮助

Linux 系统之下所配备的 vim 是 vi 的改进版本。可以使用命令 `:help` 获得在线帮助。

联机教程 `vimtutor`，声称在 25–30 分钟的时间内教会你初步应用。建议在实验课上练习一次。

直接输入

```
$ vimtutor [Return]
```

即可得到学习指导。该学习指导，可能是中文，也可能是英文，取决于当前的操作系统环境。

如果当前的环境是中文，而你又想得到英文的指导，请输入：

```
$ LANG=C vimtutor [Return]
```

Part VIII

深入 vi 编辑器

编辑命令的组合

组合基本的编辑命令得到各种编辑动作

- 增加
- 删除
- 修改
- 移动
- 复制

举例

Backup files. CE just loves backup files, and leaves them laying around everywhere. Keep in mind, the purpose of this editor is for beginners, and these backup files may come in handy for some poor person who just erased their entire document, and saved as well! You can turn off this behavior via the personal defaults file.

CE requires a special keymap for each terminal type that will be using CE, the reason I chose to use my own keymapping code, instead of using terminfo is that simply, ...

编辑命令的组合

移动行

dd 删除一行

移动到目标处 p 贴上去

复制行

yy 复制一行到临时缓冲区

移动到目标处 p 贴上去

由 dd 或 yy 复制到临时缓冲区的内容可以多次被使用

编辑命令的组合

移动行

`dd` 删除一行

移动到目标处 `p` 贴上去

复制行

`yy` 复制一行到临时缓冲区

移动到目标处 `p` 贴上去

由 `dd` 或 `yy` 复制到临时缓冲区的内容可以多次被使用

编辑命令的组合

移动行

dd 删除一行

移动到目标处 p 贴上去

复制行

yy 复制一行到临时缓冲区

移动到目标处 p 贴上去

由 dd 或 yy 复制到临时缓冲区的内容可以多次被使用

编辑命令的组合

移动单词的位置

dw 删除一个词 $\Rightarrow$ 光标移动到恰当位置 $\Rightarrow$ p 或 P 贴上去

复制词

yw 复制一个词 $\Rightarrow$ 光标移动到恰当位置 $\Rightarrow$ p 或 P 贴上去

编辑命令的组合

移动单词的位置

dw 删除一个词 $\Rightarrow$ 光标移动到恰当位置 $\Rightarrow$ p 或 P 贴上去

复制词

yw 复制一个词 $\Rightarrow$ 光标移动到恰当位置 $\Rightarrow$ p 或 P 贴上去

更大范围的光标移动

- 到达指定行：`gg`, `1G`, `G`, `58G`
- 滚屏：`Ctrl-U` `Ctrl-D`（半屏）`Ctrl-F` `Ctrl-B`（全屏）
- 移动到配对的括号：`%`
- 按句子移动：`(`, `)`
- 按段落移动：`{`, `}`
- 搜索：使用 `/` 或者 `?` 搜索命令
- 搜索：使用 `*` 或者 `#` 搜索光标所在的单词

vi 的缓冲区

- 无名缓冲区
d c s x y 等命令删去或复制的内容自动放入此缓冲区
- 数字缓冲区
1 -- 9, 9 个数字编号的缓冲区形成 FIFO 缓冲队列
- 字母缓冲区
a -- z, 26 个字母编号的缓冲区内容不滚动
- 使用双引号作为引用缓冲区的标志
"1 "2 "3 ..."9 ; "a "b "c ..."z

vi 的缓冲区

- 无名缓冲区
d c s x y 等命令删去或复制的内容自动放入此缓冲区
- 数字缓冲区
1 -- 9, 9 个数字编号的缓冲区形成 FIFO 缓冲队列
- 字母缓冲区
a -- z, 26 个字母编号的缓冲区内容不滚动
- 使用双引号作为引用缓冲区的标志
"1 "2 "3 ..."9 ; "a "b "c ..."z

vi 的缓冲区

- 无名缓冲区
d c s x y 等命令删去或复制的内容自动放入此缓冲区
- 数字缓冲区
1 -- 9, 9 个数字编号的缓冲区形成 FIFO 缓冲队列
- 字母缓冲区
a -- z, 26 个字母编号的缓冲区内容不滚动
- 使用双引号作为引用缓冲区的标志
"1 "2 "3 ..."9 ; "a "b "c ..."z

vi 的缓冲区

- 无名缓冲区
d c s x y 等命令删去或复制的内容自动放入此缓冲区
- 数字缓冲区
1 -- 9, 9 个数字编号的缓冲区形成 FIFO 缓冲队列
- 字母缓冲区
a -- z, 26 个字母编号的缓冲区内容不滚动
- 使用双引号作为引用缓冲区的标志
"1 "2 "3 ..."9 ; "a "b "c ..."z

使用 vi 的缓冲区

- 相邻的两行复制到缓冲区 a 当中
"a2yy 或"ay2y
- 把缓冲区 b 当中的内容，粘贴到光标之后
"bp
- 把缓冲区 4 当中的内容粘贴到光标之前
"4P
- 当前位置的 3 个单词删去，存放到缓冲区 c 之中，打算以后再用
"cd3w

使用 vi 的缓冲区

- 相邻的两行复制到缓冲区 a 当中
"a2yy 或"ay2y
- 把缓冲区 b 当中的内容，粘贴到光标之后
"bp
- 把缓冲区 4 当中的内容粘贴到光标之前
"4P
- 当前位置的 3 个单词删去，存放到缓冲区 c 之中，打算以后再用
"cd3w

使用 vi 的缓冲区

- 相邻的两行复制到缓冲区 a 当中
"a2yy 或"ay2y
- 把缓冲区 b 当中的内容，粘贴到光标之后
"bp
- 把缓冲区 4 当中的内容粘贴到光标之前
"4P
- 当前位置的 3 个单词删去，存放到缓冲区 c 之中，打算以后再用
"cd3w

使用 vi 的缓冲区

- 相邻的两行复制到缓冲区 a 当中
"a2yy 或"ay2y
- 把缓冲区 b 当中的内容，粘贴到光标之后
"bp
- 把缓冲区 4 当中的内容粘贴到光标之前
"4P
- 当前位置的 3 个单词删去，存放到缓冲区 c 之中，打算以后再用
"cd3w

多重文件编辑

在启动 `vi` 时，如果以多个文件作为参数，就可以连续编辑多个文件，而不必重新启动 `vi`。

- `:n` 切换到下一个文件
- `:wn` 保存当前文件，并切换到下一个文件
- `:n!` 放弃当前文件的改变，并切换到下一个文件
- `:N` 切换到上一个文件
- `:ar` 列出在编辑队列中，有哪些文件
- `:e` 开始编辑一个新文件
- `:sp` 以多窗口的方式打开一个编辑窗
- `:wq` 保存并结束一个文件
- `:wqa` 保存并结束所有文件

多重文件编辑

在启动 `vi` 时，如果以多个文件作为参数，就可以连续编辑多个文件，而不必重新启动 `vi`。

- `:n` 切换到下一个文件
- `:wn` 保存当前文件，并切换到下一个文件
- `:n!` 放弃当前文件的改变，并切换到下一个文件
- `:N` 切换到上一个文件
- `:ar` 列出在编辑队列中，有哪些文件
- `:e` 开始编辑一个新文件
- `:sp` 以多窗口的方式打开一个编辑窗
- `:wq` 保存并结束一个文件
- `:wqa` 保存并结束所有文件

多重文件编辑

在启动 `vi` 时，如果以多个文件作为参数，就可以连续编辑多个文件，而不必重新启动 `vi`。

- `:n` 切换到下一个文件
- `:wn` 保存当前文件，并切换到下一个文件
- `:n!` 放弃当前文件的改变，并切换到下一个文件
- `:N` 切换到上一个文件
- `:ar` 列出在编辑队列中，有哪些文件
- `:e` 开始编辑一个新文件
- `:sp` 以多窗口的方式打开一个编辑窗
- `:wq` 保存并结束一个文件
- `:wqa` 保存并结束所有文件

多重文件编辑

在启动 `vi` 时，如果以多个文件作为参数，就可以连续编辑多个文件，而不必重新启动 `vi`。

- `:n` 切换到下一个文件
- `:wn` 保存当前文件，并切换到下一个文件
- `:n!` 放弃当前文件的改变，并切换到下一个文件
- `:N` 切换到上一个文件
- `:ar` 列出在编辑队列中，有哪些文件
- `:e` 开始编辑一个新文件
- `:sp` 以多窗口的方式打开一个编辑窗
- `:wq` 保存并结束一个文件
- `:wqa` 保存并结束所有文件

多重文件编辑

在启动 `vi` 时，如果以多个文件作为参数，就可以连续编辑多个文件，而不必重新启动 `vi`。

- `:n` 切换到下一个文件
- `:wn` 保存当前文件，并切换到下一个文件
- `:n!` 放弃当前文件的改变，并切换到下一个文件
- `:N` 切换到上一个文件
- `:ar` 列出在编辑队列中，有哪些文件
- `:e` 开始编辑一个新文件
- `:sp` 以多窗口的方式打开一个编辑窗
- `:wq` 保存并结束一个文件
- `:wqa` 保存并结束所有文件

多重文件编辑

在启动 `vi` 时，如果以多个文件作为参数，就可以连续编辑多个文件，而不必重新启动 `vi`。

- `:n` 切换到下一个文件
- `:wn` 保存当前文件，并切换到下一个文件
- `:n!` 放弃当前文件的改变，并切换到下一个文件
- `:N` 切换到上一个文件
- `:ar` 列出在编辑队列中，有哪些文件
- `:e` 开始编辑一个新文件
- `:sp` 以多窗口的方式打开一个编辑窗
- `:wq` 保存并结束一个文件
- `:wqa` 保存并结束所有文件

多重文件编辑

在启动 `vi` 时，如果以多个文件作为参数，就可以连续编辑多个文件，而不必重新启动 `vi`。

- `:n` 切换到下一个文件
- `:wn` 保存当前文件，并切换到下一个文件
- `:n!` 放弃当前文件的改变，并切换到下一个文件
- `:N` 切换到上一个文件
- `:ar` 列出在编辑队列中，有哪些文件
- `:e` 开始编辑一个新文件
- `:sp` 以多窗口的方式打开一个编辑窗
- `:wq` 保存并结束一个文件
- `:wqa` 保存并结束所有文件

多重文件编辑

在启动 `vi` 时，如果以多个文件作为参数，就可以连续编辑多个文件，而不必重新启动 `vi`。

- `:n` 切换到下一个文件
- `:wn` 保存当前文件，并切换到下一个文件
- `:n!` 放弃当前文件的改变，并切换到下一个文件
- `:N` 切换到上一个文件
- `:ar` 列出在编辑队列中，有哪些文件
- `:e` 开始编辑一个新文件
- `:sp` 以多窗口的方式打开一个编辑窗
- `:wq` 保存并结束一个文件
- `:wqa` 保存并结束所有文件

多重文件编辑

在启动 `vi` 时，如果以多个文件作为参数，就可以连续编辑多个文件，而不必重新启动 `vi`。

- `:n` 切换到下一个文件
- `:wn` 保存当前文件，并切换到下一个文件
- `:n!` 放弃当前文件的改变，并切换到下一个文件
- `:N` 切换到上一个文件
- `:ar` 列出在编辑队列中，有哪些文件
- `:e` 开始编辑一个新文件
- `:sp` 以多窗口的方式打开一个编辑窗
- `:wq` 保存并结束一个文件
- `:wqa` 保存并结束所有文件

使用 ex 兼容命令

在全文范围内批量修改文本时，可以使用冒号开头的 `s` 命令。

- `:%s/oldstr/newstr/g`
将所有的 `oldstr` 替换为 `newstr`。
- `:%s/oldstr/newstr/gc`
将所有的 `oldstr` 替换为 `newstr`。交互确认。
- `:s/oldstr/newstr/g`
将本行所有的 `oldstr` 替换为 `newstr`。
- `:7,23s/oldstr/newstr/g`
将 7 到 23 行所有的 `oldstr` 替换为 `newstr`。
- `:$s/oldstr/newstr/`
从当前行到最后一行，所有行的第一个 `oldstr` 替换为 `newstr`。

使用 ex 兼容命令

在全文范围内批量修改文本时，可以使用冒号开头的 `s` 命令。

- `:%s/oldstr/newstr/g`
将所有的 `oldstr` 替换为 `newstr`。
- `:%s/oldstr/newstr/gc`
将所有的 `oldstr` 替换为 `newstr`。交互确认。
- `:s/oldstr/newstr/g`
将本行所有的 `oldstr` 替换为 `newstr`。
- `:7,23s/oldstr/newstr/g`
将 7 到 23 行所有的 `oldstr` 替换为 `newstr`。
- `:$s/oldstr/newstr/`
从当前行到最后一行，所有行的第一个 `oldstr` 替换为 `newstr`。

使用 ex 兼容命令

在全文范围内批量修改文本时，可以使用冒号开头的 `s` 命令。

- `:%s/oldstr/newstr/g`
将所有的 `oldstr` 替换为 `newstr`。
- `:%s/oldstr/newstr/gc`
将所有的 `oldstr` 替换为 `newstr`。交互确认。
- `:s/oldstr/newstr/g`
将本行所有的 `oldstr` 替换为 `newstr`。
- `:7,23s/oldstr/newstr/g`
将 7 到 23 行所有的 `oldstr` 替换为 `newstr`。
- `:$s/oldstr/newstr/`
从当前行到最后一行，所有行的第一个 `oldstr` 替换为 `newstr`。

使用 ex 兼容命令

在全文范围内批量修改文本时，可以使用冒号开头的 `s` 命令。

- `:%s/oldstr/newstr/g`
将所有的 `oldstr` 替换为 `newstr`。
- `:%s/oldstr/newstr/gc`
将所有的 `oldstr` 替换为 `newstr`。交互确认。
- `:s/oldstr/newstr/g`
将本行所有的 `oldstr` 替换为 `newstr`。
- `:7,23s/oldstr/newstr/g`
将 7 到 23 行所有的 `oldstr` 替换为 `newstr`。
- `:$s/oldstr/newstr/`
从当前行到最后一行，所有行的第一个 `oldstr` 替换为 `newstr`。

使用 ex 兼容命令

在全文范围内批量修改文本时，可以使用冒号开头的 `s` 命令。

- `:%s/oldstr/newstr/g`
将所有的 `oldstr` 替换为 `newstr`。
- `:%s/oldstr/newstr/gc`
将所有的 `oldstr` 替换为 `newstr`。交互确认。
- `:s/oldstr/newstr/g`
将本行所有的 `oldstr` 替换为 `newstr`。
- `:7,23s/oldstr/newstr/g`
将 7 到 23 行所有的 `oldstr` 替换为 `newstr`。
- `:$s/oldstr/newstr/`
从当前行到最后一行，所有行的第一个 `oldstr` 替换为 `newstr`。

缩写和宏

```
:ab i18n internationalization
```

ab 意思是 abbreviate. 取消缩写的命令为 :unab.

```
:unab i18n
```

```
:map Q kyypj
```

要取消指定的宏, 使用 unmap 命令:

```
:unmap Q
```

缩写和宏

```
:ab i18n internationalization
```

ab 意思是 abbreviate. 取消缩写的命令为 :unab.

```
:unab i18n
```

```
:map Q kyypj
```

要取消指定的宏, 使用 unmap 命令:

```
:unmap Q
```

vi 的外部命令使用

- 重新得到 `shell` 环境，而不退出 `vi`
:`shell` 进入，`exit` 回到 `vi` 环境。
- 在 `vi` 内执行 `Linux` 命令
:`!< 命令名 >`
- 让命令的结果影响正在编辑的文件
把命令的结果读到正在编辑的文件中
例 :`r !cal 1 2005`
又例 :`%!sort` 对当前编辑缓冲区的内容按行排序。

定制 vi 编辑器

vi 有多种参数被改变，以此改变 vi 编辑器的工作方式。

例如：在使用 / 和 ? 命令搜索字符串时，对于大小写是否区分，可由 ignorecase 参数控制。

- :set ignorecase 不区分大小写
- :set noignorecase 区分大小写

更简洁的方式是 :set ic 和 :set noic

定制 vi 编辑器

1. 了解有哪些选项

- `:set all`
- `:set`
- `:set < 选项名 >?`

定制 vi 编辑器

2. 设定选项的方式

- 布尔选项 `:set xxx, :set noxxx`
例如, `:set ignorecase, :set noignorecase`
- 数字选项 `:set xxx=9`
例如, `:set textwidth=78`
- 字符串选项 `:set xxx=str`
例如, `:set encoding=utf8`

定制 vi 编辑器

2. 设定选项的方式

- 布尔选项 `:set xxx, :set noxxx`
例如, `:set ignorecase, :set noignorecase`
- 数字选项 `:set xxx=9`
例如, `:set textwidth=78`
- 字符串选项 `:set xxx=str`
例如, `:set encoding=utf8`

定制 vi 编辑器

2. 设定选项的方式

- 布尔选项 `:set xxx, :set noxxx`
例如, `:set ignorecase, :set noignorecase`
- 数字选项 `:set xxx=9`
例如, `:set textwidth=78`
- 字符串选项 `:set xxx=str`
例如, `:set encoding=utf8`

定制 vi 编辑器

3. 常用选项的解释

ic	ignorecase	ignore upper/lower case when searching
is	incsearch	show partial matches for a search phrase
hls	hlsearch	highlight all matching phrases
nu	number	print the line number in front of each line

定制 vi 编辑器

4. 使用资源文件设定 vi 选项

vi 资源文件名 `.exrc` (一般放在用户主目录中)
(Linux 下的 vim 版本所使用的资源文件名为 `.vimrc`)
处理中文的用户需要设定:
`set encoding=prc`

程序员的特殊需要

- 自动缩进
- 括号匹配
- 函数嵌套
- 变量定义

重点命令

- `:set ai` 设置自动缩进 `autoindent`
- `Ctrl-T`, `Ctrl-D` 输入时调整缩进
- `<`, `>` 调整代码块的缩进
- `%` 查找匹配括号
- `Ctrl-]`, `Ctrl-T` 追踪调用层次（要 `ctags` 程序配合）

几点忠告

- 没有可能，也没有必要把关于 vi 编辑器的一切细节全部掌握。
- 不是学会以后使用，而是使用当中学习。
- 只需极少的命令，即可开始使用。绝大多数命令是可以替代的，它们仅仅为了提高效率而存在。

阅读材料：

Seven habits of effective text editing, Written by Bram Moolenaar
网上资源：

The Vi Lovers Home Page, <http://thomer.com/vi/vi.html>

Part IX

Linux shell

shell 是什么？

A Unix shell is a command-line interpreter and script host that provides a traditional user interface for the Unix operating system and for Unix-like systems.

关键的问题：shell 是不是操作系统的一部分？

- 本质上，shell 是 Linux 系统上的一个应用程序
- 实际上，shell 已成为 Linux 系统的一个组成部分

shell 是什么？

A Unix shell is a command-line interpreter and script host that provides a traditional user interface for the Unix operating system and for Unix-like systems.

关键的问题：shell 是不是操作系统的一部分？

- 本质上，shell 是 Linux 系统上的一个应用程序
- 实际上，shell 已成为 Linux 系统的一个组成部分

shell 是什么？

A Unix shell is a command-line interpreter and script host that provides a traditional user interface for the Unix operating system and for Unix-like systems.

关键的问题：shell 是不是操作系统的一部分？

- 本质上，shell 是 Linux 系统上的一个应用程序
- 实际上，shell 已成为 Linux 系统的一个组成部分

shell 是什么？

A Unix shell is a command-line interpreter and script host that provides a traditional user interface for the Unix operating system and for Unix-like systems.

关键的问题：shell 是不是操作系统的一部分？

- 本质上，shell 是 Linux 系统上的一个应用程序
- 实际上，shell 已成为 Linux 系统的一个组成部分

shell 种类

- Bourne Shell 家族
 - Bourne Shell (sh)
 - Korn Shell (ksh)
 - Bourne Again Shell (bash)
- C Shell 家族
 - C Shell (csh)
 - TC Shell (tcsh)

用户可以选择不同的 shell

- 登录时的缺省 shell
- 临时换用其他 shell：直接输入其他 shell 的程序名
- 改变缺省 shell：用 chsh 命令改变用户的缺省 shell
- 不同的 shell 基本功能相似，命令语法存在轻微的差异

用户可以选择不同的 shell

- 登录时的缺省 shell
- 临时换用其他 shell：直接输入其他 shell 的程序名
- 改变缺省 shell：用 `chsh` 命令改变用户的缺省 shell
- 不同的 shell 基本功能相似，命令语法存在轻微的差异

用户可以选择不同的 shell

- 登录时的缺省 shell
- 临时换用其他 shell：直接输入其他 shell 的程序名
- 改变缺省 shell：用 chsh 命令改变用户的缺省 shell
- 不同的 shell 基本功能相似，命令语法存在轻微的差异

用户可以选择不同的 shell

- 登录时的缺省 shell
- 临时换用其他 shell：直接输入其他 shell 的程序名
- 改变缺省 shell：用 chsh 命令改变用户的缺省 shell
- 不同的 shell 基本功能相似，命令语法存在轻微的差异

shell 的功能

- ① 命令行执行
- ② 文件名替换
- ③ I/O 重定向
- ④ 管道
- ⑤ 环境控制
- ⑥ 后台处理
- ⑦ 脚本支持

标准输入，标准输出，错误输出

绝大多数 Linux 程序，被设计成这样的工作方式：

- 缺省情况下，从标准输入设备那里读取输入数据
 - 处理之后的结果，缺省情况下，发给标准输出设备
 - 处理过程中的错误信息，发送到标准错误输出设备
-
- 标准输入 (stdin)：缺省的程序输入文件
 - 标准输出 (stdout)：缺省的程序输出文件
 - 标准错误 (stderr)：接收程序出错信息的缺省文件

标准输入，标准输出，错误输出

绝大多数 Linux 程序，被设计成这样的工作方式：

- 缺省情况下，从标准输入设备那里读取输入数据
- 处理之后的结果，缺省情况下，发给标准输出设备
- 处理过程中的错误信息，发送到标准错误输出设备

- 标准输入 (stdin)：缺省的程序输入文件
- 标准输出 (stdout)：缺省的程序输出文件
- 标准错误 (stderr)：接收程序出错信息的缺省文件

重定向

- > 输出重定向符号
- >> 输出重定向符号（添加方式）
- < 输入重定向
- << 输入重定向（脚本文件的嵌入文档）

重定向

- > 输出重定向符号
- >> 输出重定向符号（添加方式）
- < 输入重定向
- << 输入重定向（脚本文件的嵌入文档）

重定向

- > 输出重定向符号
- >> 输出重定向符号（添加方式）
- < 输入重定向
- << 输入重定向（脚本文件的嵌入文档）

重定向

- > 输出重定向符号
- >> 输出重定向符号（添加方式）
- < 输入重定向
- << 输入重定向（脚本文件的嵌入文档）

重定向

- > 输出重定向符号
- >> 输出重定向符号（添加方式）
- < 输入重定向
- << 输入重定向（脚本文件的嵌入文档）

再次考察不带任何参数的 cat 命令：

cat

我们已经知道，下面命令的作用：

```
cat one > two
```

猜一猜以下命令的作用：

```
cat > two
```

再猜一猜以下命令的作用：

```
cat < one
```

再看：cat < one > two

再次考察不带任何参数的 cat 命令：

```
cat
```

我们已经知道，下面命令的作用：

```
cat one > two
```

猜一猜以下命令的作用：

```
cat > two
```

再猜一猜以下命令的作用：

```
cat < one
```

再看：

```
cat < one > two
```

再次考察不带任何参数的 `cat` 命令：

```
cat
```

我们已经知道，下面命令的作用：

```
cat one > two
```

猜一猜以下命令的作用：

```
cat > two
```

再猜一猜以下命令的作用：

```
cat < one
```

再看：`cat < one > two`

再次考察不带任何参数的 cat 命令：

```
cat
```

我们已经知道，下面命令的作用：

```
cat one > two
```

猜一猜以下命令的作用：

```
cat > two
```

再猜一猜以下命令的作用：

```
cat < one
```

再看：

```
cat < one > two
```


再次考察不带任何参数的 `cat` 命令：

```
cat
```

我们已经知道，下面命令的作用：

```
cat one > two
```

猜一猜以下命令的作用：

```
cat > two
```

再猜一猜以下命令的作用：

```
cat < one
```

再看：`cat < one > two`

管道

管道是进程之间的一种通信机制。

Linux 的设计思想之一是：复杂的任务，通过多个简单的功能模块合作解决。

所以，在 Linux 当中，有许多命令，仅仅完成相当简单的工作。

管道

管道是进程之间的一种通信机制。

Linux 的设计思想之一是：复杂的任务，通过多个简单的功能模块合作解决。

所以，在 Linux 当中，有许多命令，仅仅完成相当简单的工作。

字数统计命令 wc

计算指定文件中，文本的行数、单词数、字符数。例：

```
wc report
```

结果显示为：

```
4 30 155 report
```

如果在命令之后，不跟随指定的文件名，则默认为由标准输入设备获得输入。

字数统计命令 wc

计算指定文件中，文本的行数、单词数、字符数。例：

```
wc report
```

结果显示为：

```
4 30 155 report
```

如果在命令之后，不跟随指定的文件名，则默认为由标准输入设备获得输入。

管道操作的应用实例

想了解，目录/etc下共有多少个文件，如何做？

```
ls -l /etc
```

命令的输出有多少行，文件（连同子目录）就有多少个。

方法一

```
ls -l /etc > temp  
wc temp  
rm temp
```

方法二

```
ls -l /etc | wc -l
```

方法二，无需产生中间文件，且管道两侧的命令并发执行。

管道操作的应用实例

想了解，目录/etc下共有多少个文件，如何做？

```
ls -l /etc
```

命令的输出有多少行，文件（连同子目录）就有多少个。

方法一

```
ls -l /etc > temp  
wc temp  
rm temp
```

方法二

```
ls -l /etc | wc -l
```

方法二，无需产生中间文件，且管道两侧的命令并发执行。

管道操作的应用实例

想了解，目录/etc下共有多少个文件，如何做？

```
ls -l /etc
```

命令的输出有多少行，文件（连同子目录）就有多少个。

方法一

```
ls -l /etc > temp  
wc temp  
rm temp
```

方法二

```
ls -l /etc | wc -l
```

方法二，无需产生中间文件，且管道两侧的命令并发执行。

管道操作的应用实例

想了解，目录/etc下共有多少个文件，如何做？

```
ls -l /etc
```

命令的输出有多少行，文件（连同子目录）就有多少个。

方法一

```
ls -l /etc > temp  
wc temp  
rm temp
```

方法二

```
ls -l /etc | wc -l
```

方法二，无需产生中间文件，且管道两侧的命令并发执行。

其他的例子：当前小时数？

```
date --rfc-3339=second | cut -c 12-13
```

```
date --rfc-3339=second | cut -c 12-13 > hournow
```

其他的例子：当前小时数？

```
date --rfc-3339=second | cut -c 12-13
```

```
date --rfc-3339=second | cut -c 12-13 > hournow
```

其他的例子：当前小时数？

```
date --rfc-3339=second | cut -c 12-13
```

```
date --rfc-3339=second | cut -c 12-13 > hournow
```

文件名替换

设想，当前工作目录下，有这些文件：

letter	article	report1	report2
reportm	report99	repeat	reprint
reply	meat	at	

如果你想删除所有以“report”开头的文件，更简洁的方法是什么？

文件名替换元字符

通配符

许多 Linux 命令用文件名作为其参数。当需要对一批文件进行同一类的操作时，逐个指明文件名是不方便的。

Linux 允许使用通配符定义一种模式，用以指代所有与此模式匹配的文件。

- `?` 匹配任意单个字符
- `*` 匹配 0 到多个字符构成的串
- `[list]` 匹配所列出的字符
- `[!list]` 匹配未列出的字符

文件名替换元字符

通配符

许多 Linux 命令用文件名作为其参数。当需要对一批文件进行同一类的操作时，逐个指明文件名是不方便的。

Linux 允许使用通配符定义一种模式，用以指代所有与此模式匹配的文件。

- ? 匹配任意单个字符
- * 匹配 0 到多个字符构成的串
- [list] 匹配所列出的字符
- [!list] 匹配未列出的字符

使用文件通配符的例子

- a*
- *at
- report?
- report??
- rep??t
- rep*t
- [bdt]ark
- file[1-4]
- [!a-z]*
- *

单独使用 `*` 并不能匹配那些由 `.` 开头的隐藏文件。
用 `.*` 匹配所有的隐藏文件。

使用文件通配符的例子

- `a*`
- `*at`
- `report?`
- `report??`
- `rep??t`
- `rep*t`
- `[bdt]ark`
- `file[1-4]`
- `[!a-z]*`
- `*`

单独使用 `*` 并不能匹配那些由 `.` 开头的隐藏文件。
用 `.*` 匹配所有的隐藏文件。

显示信息的命令 echo

例：

```
echo hello there
```

将在屏幕上显示：

```
hello there
```

为了要知道 `rep*t` 将被 shell 使用文件名替换功能，展开成哪些文件，可以使用命令：

```
echo rep*t
```

显示信息的命令 echo

例：

```
echo hello there
```

将在屏幕上显示：

```
hello there
```

为了要知道 `rep*t` 将被 shell 使用文件名替换功能，展开成哪些文件，可以使用命令：

```
echo rep*t
```

警告

如果把 `rm report*`
误打成 `rm report *`
会有什么效果？

文件查找命令：find

find 命令在目录树中查找文件或子目录的位置。
可以使用文件通配符作为查找的依据。

find 命令的语法格式

find 路径名 搜索选项 动作选项

两个 find 命令的例子：

```
find . -name "*.c" -print
```

```
find . -name "*.bak" -exec rm {} \;
```

文件查找命令：find

find 命令在目录树中查找文件或子目录的位置。
可以使用文件通配符作为查找的依据。

find 命令的语法格式

find 路径名 搜索选项 动作选项

两个 find 命令的例子：

```
find . -name "*.c" -print
```

```
find . -name "*.bak" -exec rm {} \;
```

文件查找命令：find

find 命令在目录树中查找文件或子目录的位置。

可以使用文件通配符作为查找的依据。

find 命令的语法格式

find 路径名 搜索选项 动作选项

两个 find 命令的例子：

```
find . -name "*.c" -print
```

```
find . -name "*.bak" -exec rm {} \;
```

文件查找命令：find

find 命令在目录树中查找文件或子目录的位置。
可以使用文件通配符作为查找的依据。

find 命令的语法格式

find 路径名 搜索选项 动作选项

两个 find 命令的例子：

```
find . -name "*.c" -print
```

```
find . -name "*.bak" -exec rm {} \;
```


文件查找命令：find

find 命令在目录树中查找文件或子目录的位置。
可以使用文件通配符作为查找的依据。

find 命令的语法格式

find 路径名 搜索选项 动作选项

两个 find 命令的例子：

```
find . -name "*.c" -print
```

```
find . -name "*.bak" -exec rm {} \;
```

shell 元字符

有一些字符，被 shell 赋予特殊的含义，从而失去它们原有的字面意义。

已经学习过的 shell 元字符：

* ? []

> < |

空格符 回车符

转义

请看命令：

```
echo 5 > 3
```

问题：上例中的 `>` 号被 shell 解释成重定向符号。若实际打算在屏幕上显示这个符号，如何实现？

```
echo 5 \> 3
```

问题：如果实际打算在屏幕上显示 `\>` 这两个符号，又如何实现？

```
echo 5 \\> 3
```

问题：如果想实际显示 `\\>` 这样连续四个符号呢？

转义

请看命令：

```
echo 5 > 3
```

问题：上例中的 `>` 号被 shell 解释成重定向符号。若实际打算在屏幕上显示这个符号，如何实现？

```
echo 5 \> 3
```

问题：如果实际打算在屏幕上显示 `\>` 这两个符号，又如何实现？

```
echo 5 \\> 3
```

问题：如果想实际显示 `\\>` 这样连续四个符号呢？

转义

请看命令：

```
echo 5 > 3
```

问题：上例中的 `>` 号被 shell 解释成重定向符号。若实际打算在屏幕上显示这个符号，如何实现？

```
echo 5 \> 3
```

问题：如果实际打算在屏幕上显示 `\>` 这两个符号，又如何实现？

```
echo 5 \\> 3
```

问题：如果想实际显示 `\\>` 这样连续四个符号呢？

引用

转义：

```
echo 5 \> 3
```

引用：

```
echo "5 > 3"
```

或者：

```
echo '5 > 3'
```

问题：如果想实际显示\\\>这样连续四个符号呢？

```
echo '5 \\\> 3'
```

```
echo "5 \\\\\\> 5"
```

引用

转义：

```
echo 5 \> 3
```

引用：

```
echo "5 > 3"
```

或者：

```
echo '5 > 3'
```

问题：如果想实际显示\\\>这样连续四个符号呢？

```
echo '5 \\\> 3'
```

```
echo "5 \\\\\\\> 5"
```

引用

转义：

```
echo 5 \> 3
```

引用：

```
echo "5 > 3"
```

或者：

```
echo '5 > 3'
```

问题：如果想实际显示\\\>这样连续四个符号呢？

```
echo '5 \\\> 3'
```

```
echo "5 \\\\\\\> 5"
```


引用

转义：

```
echo 5 \> 3
```

引用：

```
echo "5 > 3"
```

或者：

```
echo '5 > 3'
```

问题：如果想实际显示\\\>这样连续四个符号呢？

```
echo '5 \\\> 3'
```

```
echo "5 \\\\\\> 5"
```

“转义”的双重含义

- ❶ 原来的符号（元字符）被 shell 解释成另外的含义，通过转义，恢复其字面含义。

\>，不再表示输出重定向的符号，就表示大于号本身。
命令行很长，在一行当中写不下，需要换行。直接输入回车符 [Return]，shell 将会把它解释成一行命令的结束。
可以在一行的末尾输入 \ 之后再回车开始一个新行，继续输入命令。

- ❷ 原来的符号，仅仅是字面的含义，通过转义，代表另一种特殊的含义。

如：“\a”代表响铃，“\n”代表换行，等等。

```
echo -e Hi, "\n" this is a test. [Return]
```

“转义”的双重含义

- ❶ 原来的符号（元字符）被 shell 解释成另外的含义，通过转义，恢复其字面含义。

`\>`，不再表示输出重定向的符号，就表示大于号本身。

命令行很长，在一行当中写不下，需要换行。直接输入回车符 [Return]，shell 将会把它解释成一行命令的结束。

可以在一行的末尾输入 `\` 之后再回车开始一个新行，继续输入命令。

- ❷ 原来的符号，仅仅是字面的含义，通过转义，代表另一种特殊的含义。

如：`"\a"` 代表响铃，`"\n"` 代表换行，等等。

```
echo -e Hi, "\n" this is a test. [Return]
```

“转义”的双重含义

- ① 原来的符号（元字符）被 shell 解释成另外的含义，通过转义，恢复其字面含义。
\
>, 不再表示输出重定向的符号，就表示大于号本身。
命令行很长，在一行当中写不下，需要换行。直接输入回车符 [Return]，shell 将会把它解释成一行命令的结束。
可以在一行的末尾输入 \
之后再回车开始一个新行，继续输入命令。
- ② 原来的符号，仅仅是字面的含义，通过转义，代表另一种特殊的含义。
如：“\a”代表响铃，“\n”代表换行，等等。

```
echo -e Hi, "\n" this is a test. [Return]
```

“转义”的双重含义

- ① 原来的符号（元字符）被 shell 解释成另外的含义，通过转义，恢复其字面含义。
\
>, 不再表示输出重定向的符号，就表示大于号本身。
命令行很长，在一行当中写不下，需要换行。直接输入回车符 [Return]，shell 将会把它解释成一行命令的结束。
可以在一行的末尾输入 \
之后再回车开始一个新行，继续输入命令。
- ② 原来的符号，仅仅是字面的含义，通过转义，代表另一种特殊的含义。

如：“\a”代表响铃，“\n”代表换行，等等。

```
echo -e Hi, "\n" this is a test. [Return]
```

“转义”的双重含义

- ① 原来的符号（元字符）被 shell 解释成另外的含义，通过转义，恢复其字面含义。
\
>, 不再表示输出重定向的符号，就表示大于号本身。
命令行很长，在一行当中写不下，需要换行。直接输入回车符 [Return]，shell 将会把它解释成一行命令的结束。
可以在一行的末尾输入 \
之后再回车开始一个新行，继续输入命令。
- ② 原来的符号，仅仅是字面的含义，通过转义，代表另一种特殊的含义。
如：“\a”代表响铃，“\n”代表换行，等等。

```
echo -e Hi, "\n" this is a test. [Return]
```

“转义”的双重含义

- ① 原来的符号（元字符）被 shell 解释成另外的含义，通过转义，恢复其字面含义。
\
>, 不再表示输出重定向的符号，就表示大于号本身。
命令行很长，在一行当中写不下，需要换行。直接输入回车符 [Return]，shell 将会把它解释成一行命令的结束。
可以在一行的末尾输入 \
之后再回车开始一个新行，继续输入命令。
- ② 原来的符号，仅仅是字面的含义，通过转义，代表另一种特殊的含义。
如：“\a”代表响铃，“\n”代表换行，等等。

```
echo -e Hi, "\n" this is a test. [Return]
```

“转义”的双重含义

- ① 原来的符号（元字符）被 shell 解释成另外的含义，通过转义，恢复其字面含义。
\
>, 不再表示输出重定向的符号，就表示大于号本身。
命令行很长，在一行当中写不下，需要换行。直接输入回车符 [Return]，shell 将会把它解释成一行命令的结束。
可以在一行的末尾输入 \
之后再回车开始一个新行，继续输入命令。
- ② 原来的符号，仅仅是字面的含义，通过转义，代表另一种特殊的含义。
如：“\a”代表响铃，“\n”代表换行，等等。

```
echo -e Hi, "\n" this is a test. [Return]
```


引用的用法

对比：

echo 1

2

和

echo "1

2"

引号保留了中间的空格。

未用引号括住的空间，被认为是参数之间的分隔；使用引号之后，引号对之间的内容，被视为“引用”，shell 不再另行解释。

引用的用法

对比：

echo 1 2

和

echo "1 2"

引号保留了中间的空格。

未用引号括住的空间，被认为是参数之间的分隔；使用引号之后，引号对之间的内容，被视为“引用”，shell 不再另行解释。

关于单引号和双引号之间的细微差别，请在后面留意。

对比：

```
echo "This is $HOME"  
echo 'This is $HOME'
```

对比：

```
echo *  
echo "*"   
echo '*'
```

双引号对括住的 shell 变量引用，被 shell 展开。

单引号对所括住的一切内容，均严格保持原貌。

关于单引号和双引号之间的细微差别，请在后面留意。

对比：

```
echo "This is $HOME"
```

```
echo 'This is $HOME'
```

对比：

```
echo *
```

```
echo "*"
```

```
echo '*'
```

双引号对括住的 shell 变量引用，被 shell 展开。

单引号对所括住的一切内容，均严格保持原貌。

关于单引号和双引号之间的细微差别，请在后面留意。

对比：

```
echo "This is $HOME"
```

```
echo 'This is $HOME'
```

对比：

```
echo *
```

```
echo "*"
```

```
echo '*'
```

双引号对括住的 shell 变量引用，被 shell 展开。

单引号对所括住的一切内容，均严格保持原貌。

关于单引号和双引号之间的细微差别，请在后面留意。

对比：

```
echo "This is $HOME"
```

```
echo 'This is $HOME'
```

对比：

```
echo *
```

```
echo "*"
```

```
echo '*'
```

双引号对括住的 shell 变量引用，被 shell 展开。

单引号对所括住的一切内容，均严格保持原貌。

如何把引号本身显示出来？

```
echo '"Linux" is a trademark'  
echo "\"Linux\"is a trademark"
```

小结

- shell 是操作系统中，负责用户接口的部分。
- 输入输出重定向，可以改变原始数据的来源，以及处理结果的输出目的地。
- 管道把进程之间的数据流连接起来。
- 文件名替换功能，提供通配符，方便操作。
- shell 使用转义和引用，处理具有特殊含义的元字符。

Part X

过滤程序和正则表达式

部分常用过滤命令

Linux 中的某些命令，可以对其他程序的输出，做进一步的加工处理。这部分命令，可以被认为具有“过滤”功能。

已经学习过的，具有过滤作用的命令有：

`head, tail, cut`

`wc, more, less`

分离输出：tee 命令

重定向命令，可以将 Linux 命令的输出，写入到一个文件当中。
例如：

```
ls /home > dir.list
```

希望又显示结果，又把结果存入文件，如何实现？

tee 命令可以解决：

```
ls /home | tee dir.list
```

分离输出：tee 命令

重定向命令，可以将 Linux 命令的输出，写入到一个文件当中。
例如：

```
ls /home > dir.list
```

希望又显示结果，又把结果存入文件，如何实现？

tee 命令可以解决：

```
ls /home | tee dir.list
```

分离输出：tee 命令

重定向命令，可以将 Linux 命令的输出，写入到一个文件当中。
例如：

```
ls /home > dir.list
```

希望又显示结果，又把结果存入文件，如何实现？

tee 命令可以解决：

```
ls /home | tee dir.list
```

排序命令: sort

sort 命令既可以对文件的内容排序，又可以对其他命令的输出排序。排序之后的内容，可以写入文件，也可送标准输出。

注意以下有用的排序选项：

- b 忽略前导空格
- d 用字典顺序排序
- f 忽略大小写的区别
- n 数字以数值大小排序
- r 逆序（缺省为升序）
- o 输出改为指定文件，而非标准输出设备
- k 指定排序的关键字位置

排序命令: sort

sort 命令既可以对文件的内容排序，又可以对其他命令的输出排序。排序之后的内容，可以写入文件，也可送标准输出。

注意以下有用的排序选项：

- b 忽略前导空格
- d 用字典顺序排序
- f 忽略大小写的区别
- n 数字以数值大小排序
- r 逆序（缺省为升序）
- o 输出改为指定文件，而非标准输出设备
- k 指定排序的关键字位置

某些文件是“结构性的”。

每一行为一个记录，每个记录含若干字段，字段之间由空格或制表符（Tab）分隔。

请看电话号码文件的例子。

考虑如何按文件的大小，显示一个目录中的文件。怎样用 `sort` 命令作为过滤，来实现？

提示：`-n` 选项，`-k` 选项

某些文件是“结构性的”。

每一行为一个记录，每个记录含若干字段，字段之间由空格或制表符（Tab）分隔。

请看电话号码文件的例子。

考虑如何按文件的大小，显示一个目录中的文件。怎样用 `sort` 命令作为过滤，来实现？

提示：`-n` 选项，`-k` 选项

某些文件是“结构性的”。

每一行为一个记录，每个记录含若干字段，字段之间由空格或制表符（Tab）分隔。

请看电话号码文件的例子。

考虑如何按文件的大小，显示一个目录中的文件。怎样用 `sort` 命令作为过滤，来实现？

提示：`-n` 选项，`-k` 选项

滤除重复行：uniq

经过 uniq 的过滤，**相邻的**重复行，仅仅显示一次。

uniq 还有滤掉空行的功能。

问题：如何滤掉所有的重复的行（包括那些**不相邻的**？）

滤除重复行：uniq

经过 uniq 的过滤，**相邻的**重复行，仅仅显示一次。

uniq 还有滤掉空行的功能。

问题：如何滤掉所有的重复的行（包括那些**不相邻的**？）

搜索命令：grep

grep 命令可以从一个或一批文件之内容中，搜索含有特定字符串的行。

例：一大堆文件中，哪一个是我写给 Tom 的信？

例：在一份电话号码簿文件中查，38976542 是谁的电话？

更复杂的查找，需要使用“正则表达式”，来描述被搜索的字符串。

regex: Regular Expression (Global Regular Expression Print)

搜索命令：grep

grep 命令可以从一个或一批文件之内容中，搜索含有特定字符串的行。

例：一大堆文件中，哪一个是我写给 Tom 的信？

例：在一份电话号码簿文件中查，38976542 是谁的电话？

更复杂的查找，需要使用“正则表达式”，来描述被搜索的字符串。

regex: Regular Expression (Global Regular Expression Print)

搜索命令：grep

grep 命令可以从一个或一批文件之内容中，搜索含有特定字符串的行。

例：一大堆文件中，哪一个是我写给 Tom 的信？

例：在一份电话号码簿文件中查，38976542 是谁的电话？

更复杂的查找，需要使用“正则表达式”，来描述被搜索的字符串。

regex: Regular Expression (Global Regular Expression Print)

搜索命令：grep

grep 命令可以从一个或一批文件之内容中，搜索含有特定字符串的行。

例：一大堆文件中，哪一个是我写给 Tom 的信？

例：在一份电话号码簿文件中查，38976542 是谁的电话？

更复杂的查找，需要使用“正则表达式”，来描述被搜索的字符串。

regex: Regular Expression (Global Regular Expression Print)

最简单的 grep 命令

`grep < 字符串模式 > < 接受搜索的文件名 >`

以上命令执行后，如果文件中含有与“字符串模式”相匹配的行，该行就会被显示出来。

例：

```
grep monitor /etc/x11/XF86Config
```

其中的字符串模式，最好放到引号对中间。（为什么？）

最简单的 grep 命令

`grep < 字符串模式 > < 接受搜索的文件名 >`

以上命令执行后，如果文件中含有与“字符串模式”相匹配的行，该行就会被显示出来。

例：

```
grep monitor /etc/x11/XF86Config
```

其中的字符串模式，最好放到引号对中间。（为什么？）

grep 命令的常用选项

- c 显示每个文件中匹配的行数
- i 不分大小写
- l 显示具有相匹配行的文件名，而不是匹配行本身
- n 显示匹配行的行号
- v 显示不匹配的行（匹配的行不显示）
- r 搜索范围包含以下各级子目录中的文件

应用实例

1. 我写的一封信，只记得是写给 David 的，文件名忘记了，保存在什么地方也忘记了，但一定在我本人的主目录或者下级的某个子目录中，文件的内容里，一定有 David 这个词。如何找到那份文件？

先进入用户的主目录，再执行：

```
grep -ir david .
```

应用实例

1. 我写的一封信，只记得是写给 David 的，文件名忘记了，保存在什么地方也忘记了，但一定在我本人的主目录或者下级的某个子目录中，文件的内容里，一定有 David 这个词。如何找到那份文件？

先进入用户的主目录，再执行：

```
grep -ir david .
```

2. 我在某一个幻灯片文件中，谈到有关 more 命令的格式问题？

```
grep 'more' /mnt/sda1/unixabc/*.tex
```

输出的内容太多，可以再筛选一次：

```
grep 'more' /mnt/sda1/unixabc/*.tex | grep '格式'
```

2. 我在某一个幻灯片文件中，谈到有关 more 命令的格式问题？

```
grep 'more' /mnt/sda1/unixabc/*.tex
```

输出的内容太多，可以再筛选一次：

```
grep 'more' /mnt/sda1/unixabc/*.tex | grep '格式'
```

2. 我在某一个幻灯片文件中，谈到有关 more 命令的格式问题？

```
grep 'more' /mnt/sda1/unixabc/*.tex
```

输出的内容太多，可以再筛选一次：

```
grep 'more' /mnt/sda1/unixabc/*.tex | grep '格式'
```


3. 哪一个命令可以用来对软盘格式化？

`man -k disk` , 输出结果太多, 不知所云!

`man -k disk | grep '(1'`

3. 哪一个命令可以用来对软盘格式化？

`man -k disk` , 输出结果太多, 不知所云!

`man -k disk | grep '(1'`

3. 哪一个命令可以用来对软盘格式化？

`man -k disk` , 输出结果太多, 不知所云!

`man -k disk | grep '(1'`

在 grep 命令中应用正则表达式

grep 命令不仅可以搜索一个固定的字符串，更重要的，还可以使用“正则表达式”搜索一种模式。

如：

```
grep "h[eu]llo" myfile
```

或者：

```
grep "colou?r" myfile
```

虽然正则表达式与文件名通配符的规则有相仿之处，其实两者很不相同。

使用正则表达式的命令或实用程序还有：

vi, awk, sed, perl ...

在 grep 命令中应用正则表达式

grep 命令不仅可以搜索一个固定的字符串，更重要的，还可以使用“正则表达式”搜索一种模式。

如：

```
grep "h[eu]llo" myfile
```

或者：

```
grep "colou?r" myfile
```

虽然正则表达式与文件名通配符的规则有相仿之处，其实两者很不相同。

使用正则表达式的命令或实用程序还有：

vi, awk, sed, perl ...

正则表达式

Regular Expression

正则表达式的起源

最早，两位神经生理学家 Warren McCulloch 和 Walter Pitts 研究出一种数学方式来描述神经网络。

1956 年，一位叫 Stephen Kleene 的数学家在 McCulloch 和 Pitts 早期工作的基础上，发表了一篇标题为“神经网络事件的表示法”的论文，引入了正则表达式的概念。正则表达式就是用来描述他称为“正则集的代数”的表达式，因此采用“正则表达式”这个术语。

随后，Ken Thompson 将此方法用于搜索算法的一些早期研究，Ken Thompson 是 Unix 的主要发明人。正则表达式的第一个实用应用程序就是 Unix 中的 `qed` 编辑器。

正则表达式的定义

RE 描述了一种字符串匹配的模式 (pattern)，可以用来检查一个串是否含有某种子串、将匹配的子串做替换或者从某个串中取出符合某个条件的子串等。

shell 中的文件通配符也使用了 RE 使用的 *, ?, [] 等符号。但和正则表达式的意义不同。

(Linux 当中存在的不一致性)

正则表达式的构成

RE 由以下主要元素构成：

- ① 字符
- ② 字符集
- ③ 限定符
- ④ 定位符
- ⑤ 选择符
- ⑥ 引用

字符

普通字符：A, B, ...a, b ...1, 2 ...
_ (下划线)、_ (空格) ...

特殊字符（不可打印字符）：换行符：\n, 回车符：\r, 制表符：
\t, ...
元字符：

. 任意单个字符（除换行符外）
^ 行的开头（或者表示逻辑非）
\$ 行的结尾
\ 转义字符：
 \\$ 代表 \$
 \^ 代表 ^
 \\ 代表 \ 自身。

字符

普通字符：A, B, ...a, b ...1, 2 ...
_ (下划线)、_ (空格) ...

特殊字符（不可打印字符）：换行符：\n, 回车符：\r, 制表符：
\t, ...
元字符：

. 任意单个字符（除换行符外）
^ 行的开头（或者表示逻辑非）
\$ 行的结尾
\ 转义字符：
 \\$ 代表 \$
 \^ 代表 ^
 \\ 代表 \ 自身。

字符

普通字符：A, B, ...a, b ...1, 2 ...
_ (下划线)、_ (空格) ...

特殊字符（不可打印字符）：换行符：\n, 回车符：\r, 制表符：
\t, ...
元字符：

. 任意单个字符（除换行符外）
^ 行的开头（或者表示逻辑非）
\$ 行的结尾
\ 转义字符：
 \\$ 代表 \$
 \^ 代表 ^
 \\ 代表 \ 自身。

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “`once upon a time`” 匹配

与 “`There once was a man from New York`” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “`Who kept all of this cash in a bucket`” 匹配

与 “`buckets`” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

`There once was a man from New York`

`Who kept all of his cash in a bucket.`

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “`once upon a time`” 匹配

与 “`There once was a man from New York`” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “`Who kept all of this cash in a bucket`” 匹配

与 “`buckets`” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

`There once was a man from New York`

`Who kept all of his cash in a bucket.`

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “once upon a time” 匹配

与 “There once was a man from New York” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “Who kept all of this cash in a bucket” 匹配

与 “buckets” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

There once was a man from New York

Who kept all of his cash in a bucket.

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “once upon a time” 匹配

与 “There once was a man from New York” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “Who kept all of this cash in a bucket” 匹配

与 “buckets” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

There once was a man from New York

Who kept all of his cash in a bucket.

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “`once upon a time`” 匹配

与 “`There once was a man from New York`” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “`Who kept all of this cash in a bucket`” 匹配

与 “`buckets`” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

There once was a man from New York

Who kept all of his cash in a bucket.

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “`once upon a time`” 匹配

与 “`There once was a man from New York`” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “`Who kept all of this cash in a bucket`” 匹配

与 “`buckets`” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

There once was a man from New York
Who kept all of his cash in a bucket.

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “`once upon a time`” 匹配

与 “`There once was a man from New York`” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “`Who kept all of this cash in a bucket`” 匹配

与 “`buckets`” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

```
There once was a man from New York
Who kept all of his cash in a bucket.
```

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “`once upon a time`” 匹配

与 “`There once was a man from New York`” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “`Who kept all of this cash in a bucket`” 匹配

与 “`buckets`” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

```
There once was a man from New York  
Who kept all of his cash in a bucket.
```

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “`once upon a time`” 匹配

与 “`There once was a man from New York`” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “`Who kept all of this cash in a bucket`” 匹配

与 “`buckets`” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

```
There once was a man from New York  
Who kept all of his cash in a bucket.
```

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “`once upon a time`” 匹配

与 “`There once was a man from New York`” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “`Who kept all of this cash in a bucket`” 匹配

与 “`buckets`” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

```
There once was a man from New York
Who kept all of his cash in a bucket.
```

举例：

`^once` 包含特殊字符 `^`，只匹配那些以 `once` 开头的字符串。

与字符串 “`once upon a time`” 匹配

与 “`There once was a man from New York`” 不匹配

`$` 符号匹配以给定模式结尾的字符串。

`bucket$` 与 “`Who kept all of this cash in a bucket`” 匹配

与 “`buckets`” 不匹配

字符 `^` 和 `$` 同时使用时，表示精确匹配。例如：

`^bucket$` 匹配什么？

最普通的例子：

`once` 与以下的字符串如何匹配？

```
There once was a man from New York
Who kept all of his cash in a bucket.
```

字符集

字符集匹配集合中的任一字符。方括号 `[]` 是字符集的标识。
怎样表示方括号自身呢？

`[AaEeIiOoUu]` 匹配任何单个元音字符

`[tT]he` 匹配 “the” 和 “The”

`^` 字符用在方括号内的首位时，表示 “NOT”，如：`[^Tt]` 代表除 `T` 或 `t` 之外的其他单个字符

字符集

字符集匹配集合中的任一字符。方括号 `[]` 是字符集的标识。
怎样表示方括号自身呢？

`[AaEeIiOoUu]` 匹配任何单个元音字符

`[tT]he` 匹配 “the” 和 “The”

`^` 字符用在方括号内的首位时，表示 “NOT”，如：`[^Tt]` 代表除 `T` 或 `t` 之外的其他单个字符

字符集

字符集匹配集合中的任一字符。方括号 `[]` 是字符集的标识。
怎样表示方括号自身呢？

`[AaEeIiOoUu]` 匹配任何单个元音字符

`[tT]he` 匹配 “the” 和 “The”

`^` 字符用在方括号内的首位时，表示 “NOT”，如：`[^Tt]` 代表除 `T` 或 `t` 之外的其他单个字符

字符集

字符集匹配集合中的任一字符。方括号 `[]` 是字符集的标识。
怎样表示方括号自身呢？

`[AaEeIiOoUu]` 匹配任何单个元音字符

`[tT]he` 匹配 “the” 和 “The”

`^` 字符用在方括号内的首位时，表示 “NOT”，如：`[^Tt]` 代表除 `T` 或 `t` 之外的其他单个字符

可以用指定范围的方式，表示字符集：

[a-e] 等效于 [abcde]

[a-z] 所有的小写字母

[A-Z] 所有的大写字母

[a-zA-Z] 所有的字母

[0-9] 所有的数字

[0-9\.\-] 所有的数字，句号和减号

[\f\r\t\n] 所有的白字符

字符集只匹配单个字符

`^[a-z][0-9]$` 匹配 “z2”，“t6”，“g7”，不匹配 “ab2”，“r2d3”。

`^[a-z][a-z][0-9]$` 可以匹配 “ab2”。

可以用指定范围的方式，表示字符集：

[a-e] 等效于 [abcde]

[a-z] 所有的小写字母

[A-Z] 所有的大写字母

[a-zA-Z] 所有的字母

[0-9] 所有的数字

[0-9\.\-] 所有的数字，句号和减号

[\f\r\t\n] 所有的白字符

字符集只匹配单个字符

`^[a-z][0-9]$` 匹配 “z2”，“t6”，“g7”，不匹配 “ab2”，“r2d3”。

`^[a-z][a-z][0-9]$` 可以匹配 “ab2”。

限定符

限定符指明在其前面的字符或子表达式出现的次数。

* 之前的字符或子表达式重复零次或多次

zo* 匹配 z, zo, zoo, zooo

+ 一次或多次

zo+ 匹配 zo, zoo, zooo
 不匹配 z

? 零次或一次

do(es)? 匹配 do 或 does

does? 匹配 doe 或 does

限定符

限定符指明在其前面的字符或子表达式出现的次数。

* 之前的字符或子表达式重复零次或多次

zo* 匹配 z, zo, zoo, zooo

+ 一次或多次

zo+ 匹配 zo, zoo, zooo

不匹配 z

? 零次或一次

do(es)? 匹配 do 或 does

does? 匹配 doe 或 does

限定符

限定符指明在其前面的字符或子表达式出现的次数。

* 之前的字符或子表达式重复零次或多次

zo* 匹配 z, zo, zoo, zooo

+ 一次或多次

zo+ 匹配 zo, zoo, zooo

不匹配 z

? 零次或一次

do(es)? 匹配 do 或 does

does? 匹配 doe 或 does

限定符（续）

$\{n\}$ 匹配确定的 n 次
 $o\{2\}$ 不匹配 Bob 中的 o
 匹配 food 中的两个 o

$\{n,\}$ 至少 n 次

$\{,m\}$ 至多 m 次

$\{n,m\}$ n 到 m 次

$*$ 等价于 $\{0,\}$

$+$ 等价于 $\{1,\}$

$?$ 等价于 $\{0,1\}$

限定符（续）

$\{n\}$ 匹配确定的 n 次
 $o\{2\}$ 不匹配 Bob 中的 o
 匹配 food 中的两个 o

$\{n,\}$ 至少 n 次

$\{,m\}$ 至多 m 次

$\{n,m\}$ n 到 m 次

$*$ 等价于 $\{0,\}$

$+$ 等价于 $\{1,\}$

$?$ 等价于 $\{0,1\}$

定位符

定位符指明字符或子表达式所在的位置。

比如：

there were many orchards of fruit tree

in the dark it was like summer lightning

希望匹配单词“the” 不希望匹配“there” 当中的前三个字母。

可以使用模式 `the` （在单词的后面有一个空格）。

但这样仍可能误匹配单词“breathe”（`the` 后面也有一个空格）

定位符（续）

<code>^</code>	:	行首
<code>\$</code>	:	行尾
<code>\<</code>	:	单词的开始处
<code>\></code>	:	单词的结尾处
<code>\b</code>	:	单词的边界（首或尾）
<code>\B</code>	:	不在单词的边界

例如：`\<the\>` 只匹配单词 “the”

`\<it\>` 只匹配单词 “it”，不匹配 “sitting” 当中的 “it”。

选择

圆括号 `()` 表示一个组，某些应用场合中用 `\(` 和 `\)` 代替
竖线 `|` 表示 “OR”，用来表示选择的含义。

例：

`ab+c` 表示 `a` 后面有一个 `b` 或多个 `b` 并跟有 `c`
`yabcbw`, `abbc57` 符合

`(ab)+c` 表示一个或多个字符串 `ab` 并跟有 `c`
`zabcd`, `ababc` 符合

`industr(y|ies)` 相当于 `industry|industries`
匹配 `industry`, 也匹配 `industries`

选择

圆括号 `()` 表示一个组，某些应用场合中用 `\(` 和 `\)` 代替
竖线 `|` 表示 “OR”，用来表示选择的含义。

例：

`ab+c` 表示 `a` 后面有一个 `b` 或多个 `b` 并跟有 `c`
`yabcbw`, `abbc57` 符合

`(ab)+c` 表示一个或多个字符串 `ab` 并跟有 `c`
`zabcd`, `ababc` 符合

`industr(y|ies)` 相当于 `industry|industries`
匹配 `industry`, 也匹配 `industries`

引用

& : 整个正则表达式所匹配的内容
\n : 第 n 个子表达式所匹配的内容
 \1 第一个子表达式匹配的内容
 \2 第二个子表达式匹配的内容

正则表达式的例子

用一个例子来帮助理解“引用”的概念。
例如，vim 正在编辑的文件中有两行：

```
025  nanjing
027  wuhan
```

尝试命令：

```
:%s/0[0-9]* \+[a-zA-Z]\+/X&Y/
:%s/\(0[0-9]*\) \+\[a-zA-Z]\+\)/\2 \1/
```

学习正则表达式的途径

最好的方法是在 vim (vi) 环境当中多加练习。

使用 `/pattern` 或者 `:%s/pattern/replacement/gc` 命令细心实验观察。

其他的应用

有些 Windows 操作系统上运行的编辑器，如 UltraEditor，允许使用正则表达式进行搜索。

PHP，JAVA，JAVAScript 和 VBScript 当中，都有正则表达式的应用。

Part XI

Linux shell（续）

命令替换：元字符：前重音符号 “`”

- 有两种命令替换的表达表达方式
- ``command``
- `$(command)`
- 其作用是用命令执行的结果替换原始内容
- 例子：

```
cp -r mywork backup/`date -R | cut -d" " -f5`
```

命令序列：元字符：分号 “;”

- 同一个命令行中的多个命令，用分号分隔
- 被分号隔开的命令，会依次执行
- `cmd1; cmd2; cmd3; cmd4`
- 例子：
`download; build; install; test`

命令编组：括号对 ()

圆括号对命令编组。

```
( ls; date; pwd ) > outfile
```

对比：

```
ls; date; pwd    > outfile
```

后台执行：元字符 “&”

如果命令行的尾部，用 & 符号结束，表示，该命令放到后台执行。

例：

```
find . -name xyz.xyz -print > result &  
(date; find / | sort ; date) 2> /dev/null > result &
```

思考：什么时候需要将作业放到后台执行？什么样的作业适合放到后台执行？

后台作业

- 耗时较长
- 运行时无交互
- 数据往往来源于文件
- 结果一般输出到文件

作业控制

- 命令行末尾，加 & 字符，作业放到后台执行
- 组合键 [Ctrl-Z]：把正在前台执行的作业挂起，并置于后台
- fg 命令：把后台作业调到前台执行
fg <作业号>
- bg 命令：让后台挂起的作业，继续在后台执行
bg <作业号>
- jobs 命令：了解当前作业情况

作业控制

- 命令行末尾，加 & 字符，作业放到后台执行
- 组合键 [Ctrl-Z]：把正在前台执行的作业挂起，并置于后台
- fg 命令：把后台作业调到前台执行
fg <作业号>
- bg 命令：让后台挂起的作业，继续在后台执行
bg <作业号>
- jobs 命令：了解当前作业情况

作业控制

- 命令行末尾，加 & 字符，作业放到后台执行
- 组合键 [Ctrl-Z]：把正在前台执行的作业挂起，并置于后台
- fg 命令：把后台作业调到前台执行
fg <作业号>
- bg 命令：让后台挂起的作业，继续在后台执行
bg <作业号>
- jobs 命令：了解当前作业情况

作业控制

- 命令行末尾，加 & 字符，作业放到后台执行
- 组合键 [Ctrl-Z]：把正在前台执行的作业挂起，并置于后台
- fg 命令：把后台作业调到前台执行
fg <作业号>
- bg 命令：让后台挂起的作业，继续在后台执行
bg <作业号>
- jobs 命令：了解当前作业情况

作业控制

- 命令行末尾，加 & 字符，作业放到后台执行
- 组合键 [Ctrl-Z]：把正在前台执行的作业挂起，并置于后台
- fg 命令：把后台作业调到前台执行
fg <作业号>
- bg 命令：让后台挂起的作业，继续在后台执行
bg <作业号>
- jobs 命令：了解当前作业情况

作业控制（续）

- 终止后台作业
kill <作业号>
- 终止前台作业
[Ctrl-c]
- 要想在用户退出后，作业继续运行
nohup <想要继续执行的命令>

作业控制（续）

- 终止后台作业
kill <作业号>
- 终止前台作业
[Ctrl-c]
- 要想在用户退出后，作业继续运行
nohup <想要继续执行的命令>

作业控制（续）

- 终止后台作业
kill <作业号>
- 终止前台作业
[Ctrl-c]
- 要想在用户退出后，作业继续运行
nohup <想要继续执行的命令>

进程状态显示

- 进程状态显示命令：ps
- Linux 系统中的 ps 命令，接受三种风格的选项：
- UNIX 风格：用一个减号引导，选项为单个字符
- BSD 风格：不用减号引导，选项为单个字符
- GNU 风格：用两个减号引导，选项为单词

进程状态显示

- 进程状态显示命令：ps
- Linux 系统中的 ps 命令，接受三种风格的选项：
- UNIX 风格：用一个减号引导，选项为单个字符
- BSD 风格：不用减号引导，选项为单个字符
- GNU 风格：用两个减号引导，选项为单词

进程状态显示

- 进程状态显示命令：ps
- Linux 系统中的 ps 命令，接受三种风格的选项：
- UNIX 风格：用一个减号引导，选项为单个字符
- BSD 风格：不用减号引导，选项为单个字符
- GNU 风格：用两个减号引导，选项为单词

进程状态显示

- 进程状态显示命令：ps
- Linux 系统中的 ps 命令，接受三种风格的选项：
- UNIX 风格：用一个减号引导，选项为单个字符
- BSD 风格：不用减号引导，选项为单个字符
- GNU 风格：用两个减号引导，选项为单词

进程状态显示

- 进程状态显示命令：ps
- Linux 系统中的 ps 命令，接受三种风格的选项：
- UNIX 风格：用一个减号引导，选项为单个字符
- BSD 风格：不用减号引导，选项为单个字符
- GNU 风格：用两个减号引导，选项为单词

ps 命令的常用选项

- UNIX 风格的常用选项：

`ps -ef`

-e 代表选择所有进程

-f 代表完整的输出格式

- BSD 风格的常用选项：

`ps auxw`

a 代表选择所有进程

u 代表显示出进程所有者

x 代表包括不拥有终端的进程

w 代表输出更详尽的信息

ps 命令的常用选项

- UNIX 风格的常用选项：

ps -ef

-e 代表选择所有进程

-f 代表完整的输出格式

- BSD 风格的常用选项：

ps auxw

a 代表选择所有进程

u 代表显示出进程所有者

x 代表包括不拥有终端的进程

w 代表输出更详尽的信息

进程控制

- 终止进程
kill 进程号
- 对顽固的进程，发出不同信号
kill -信号值 进程号
- 了解系统支持哪些信号值
kill -l

进程控制

- 终止进程
kill 进程号
- 对顽固的进程，发出不同信号
kill -信号值 进程号
- 了解系统支持哪些信号值
kill -l

进程控制

- 终止进程
kill 进程号
- 对顽固的进程，发出不同信号
kill -信号值 进程号
- 了解系统支持哪些信号值
kill -l

shell 变量

为了具有一定的适应性，shell 需要一些信息构成运行环境。

例如：当前用户名（LOGIN 或 USER），

用户主目录（HOME）。

通常，为用户的应用程序，提供关于运行环境信息的那些变量，被成为环境变量。

shell 的常用环境变量

- HOME 用户主目录
- DISPLAY 当前显示器
- PWD 当前目录
- SHELL 用户缺省 shell
- TERM 终端类型
- PATH 搜索路径
- USER 用户名
- ...

显示、设置、清除、引用 shell 变量

<code>set</code>	显示全部的 <code>shell</code> 变量
<code>XYZ=10</code>	设置 <code>XYZ</code> 变量的值为 10
<code>unset XYZ</code>	清除 <code>XYZ</code> 变量
<code>echo \$XYZ</code>	引用 <code>XYZ</code> 变量的例子，变量之前的 <code>\$</code> 符号不能缺少

shell 的配置

对 shell 的配置是通过配置文件进行的。

- 全局配置文件 `/etc/profile`
- 用户配置文件 `$HOME/.profile`

(实际的情况要更加复杂 ...)

Linux 缺省的 shell 是 `bash`，`bash` 的配置文件，是用户主目录之下的隐藏文件：`.bash_profile`

(可以用波浪号 `~` 代替 `$HOME`，也即用户主目录)

shell 的配置

对 shell 的配置是通过配置文件进行的。

- 全局配置文件 `/etc/profile`
- 用户配置文件 `$HOME/.profile`

(实际的情况要更加复杂 ...)

Linux 缺省的 shell 是 `bash` , `bash` 的配置文件, 是用户主目录之下的隐藏文件: `.bash_profile`

(可以用波浪号 `~` 代替 `$HOME`, 也即用户主目录)

Part XII

shell 编程

shell 脚本

shell 脚本是一个文件，其中包含将由 shell 解释并执行的一系列命令。这些命令有的由 shell 直接执行，另一些由 shell 交给其他的实用程序执行。

Linux 的 shell 脚本是一种将 Linux 中的实用程序组合起来解决复杂问题的有效手段。

一个脚本的实例

要了解我的用户主目录中，究竟是哪些文件占用了太多的磁盘空间，可以使用命令：

```
ls -l ~ | sort -k 5 -n -r | head -n 3
```

这条命令太长，使用时，容易出现打字错误。
将这条命令写到一个文件中，文件的名字为：

bigfile

以后在命令行上直接输入

./bigfile

或者：

sh bigfile

就可以执行所需要的命令。

一个脚本的实例

要了解我的用户主目录中，究竟是哪些文件占用了太多的磁盘空间，可以使用命令：

```
ls -l ~ | sort -k 5 -n -r | head -n 3
```

这条命令太长，使用时，容易出现打字错误。
将这条命令写到一个文件中，文件的名字为：

bigfile

以后在命令行上直接输入

./bigfile

或者：

sh bigfile

就可以执行所需要的命令。

调用脚本

显式地将脚本文件提交给特定的 shell 执行。

格式：

<shell 程序名> <脚本文件名>

例如：

```
sh myscript
```

```
/bin/sh myscript
```

```
bash myscript
```

```
bash /home/david/myscript
```

```
/bin/csh /home/david/myscript
```

调用脚本

显式地将脚本文件提交给特定的 shell 执行。

格式：

<shell 程序名> <脚本文件名>

例如：

```
sh myscript
```

```
/bin/sh myscript
```

```
bash myscript
```

```
bash /home/david/myscript
```

```
/bin/csh /home/david/myscript
```

执行脚本

先为要执行的脚本文件加上可执行权限：

```
chmod +x myscript
```

直接输入脚本文件名，即可执行该脚本：

```
./myscript
```

```
/home/david/myscript
```

如果不指明路径，系统自动从 PATH 环境变量中，寻找程序文件的存放位置。

一般情况下，PATH 环境变量中，不包含当前的工作目录，所以，脚本文件之前要特别指明。

执行脚本

先为要执行的脚本文件加上可执行权限：

```
chmod +x myscript
```

直接输入脚本文件名，即可执行该脚本：

```
./myscript
```

```
/home/david/myscript
```

如果不指明路径，系统自动从 PATH 环境变量中，寻找程序文件的存放位置。

一般情况下，PATH 环境变量中，不包含当前的工作目录，所以，脚本文件之前要特别指明。

执行脚本

先为要执行的脚本文件加上可执行权限：

```
chmod +x myscript
```

直接输入脚本文件名，即可执行该脚本：

```
./myscript
```

```
/home/david/myscript
```

如果不指明路径，系统自动从 PATH 环境变量中，寻找程序文件的存放位置。

一般情况下，PATH 环境变量中，不包含当前的工作目录，所以，脚本文件之前要特别指明。

脚本内容

- 脚本中的每一行，是一个 Linux 的命令行
- 井号 “#” 是表示注释的符，从井号开始到本行末，是注释内容
- 第一行的用井号与感叹号开头的行，专为指明运行此脚本的 shell。例如：

`#!/bin/sh` 或者 `#!/bin/bash` 或者 `#!/bin/csh`

如果没有这样一行，系统将会使用“缺省 shell”，来运行脚本程序。

- shell 有专门的控制命令，控制脚本中命令的执行次序：顺序，分支，循环

脚本内容

- 脚本中的每一行，是一个 Linux 的命令行
- 井号 “#” 是表示注释的符，从井号开始到本行末，是注释内容
- 第一行的用井号与感叹号开头的行，专为指明运行此脚本的 shell。例如：

`#!/bin/sh` 或者 `#!/bin/bash` 或者 `#!/bin/csh`

如果没有这样一行，系统将会使用“缺省 shell”，来运行脚本程序。

- shell 有专门的控制命令，控制脚本中命令的执行次序：顺序，分支，循环

脚本内容

- 脚本中的每一行，是一个 Linux 的命令行
- 井号 “#” 是表示注释的符，从井号开始到本行末，是注释内容
- 第一行的用井号与感叹号开头的行，专为指明运行此脚本的 shell。例如：

`#!/bin/sh` 或者 `#!/bin/bash` 或者 `#!/bin/csh`

如果没有这样一行，系统将会使用“缺省 shell”，来运行脚本程序。

- shell 有专门的控制命令，控制脚本中命令的执行次序：顺序，分支，循环

脚本内容

- 脚本中的每一行，是一个 Linux 的命令行
- 井号 “#” 是表示注释的符，从井号开始到本行末，是注释内容
- 第一行的用井号与感叹号开头的行，专为指明运行此脚本的 shell。例如：

`#!/bin/sh` 或者 `#!/bin/bash` 或者 `#!/bin/csh`

如果没有这样一行，系统将会使用“缺省 shell”，来运行脚本程序。

- shell 有专门的控制命令，控制脚本中命令的执行次序：顺序，分支，循环

shell 编程基础：变量

shell 脚本中的变量使用前不需要专门说明。如：

```
NAME=David
```

就定义了变量 NAME。

shell 变量只有一种类型：字符串

引用变量在变量名之前加 “\$”，如：

```
echo $NAME
```

清除变量，用 unset 命令：

```
unset NAME
```

shell 编程基础：变量

shell 脚本中的变量使用前不需要专门说明。如：

```
NAME=David
```

就定义了变量 NAME。

shell 变量只有一种类型：字符串

引用变量在变量名之前加 “\$”，如：

```
echo $NAME
```

清除变量，用 `unset` 命令：

```
unset NAME
```

shell 编程基础：变量

shell 脚本中的变量使用前不需要专门说明。如：

```
NAME=David
```

就定义了变量 NAME。

shell 变量只有一种类型：字符串

引用变量在变量名之前加 “\$”，如：

```
echo $NAME
```

清除变量，用 unset 命令：

```
unset NAME
```

读取输入

shell 脚本使用 read 命令从标准输入设备读取字符串。

```
echo  
echo "Input your name:"  
read name  
echo "Your name is $name"  
echo
```

命令行参数

shell 脚本允许在命令行上输入选项和参数。

shell 脚本可以从命令行读入最多 10 个参数。它们被自动地置入“位置变量”当中。这些位置变量将在脚本中被使用。

- \$0 : 命令行上所键入的脚本文件名
- \$1,\$2,...,\$9 : 脚本文件名之后的第 1 到第 9 个参数
- \$# : 命令行参数的个数
- \$@ : 所有命令行参数
(每个参数以引号形式保存)
- \$* : 所有命令行参数
(保存在单一的字符串中)
- \$? : 最后一个命令的退出状态 (返回码)
- \$\$: 正在执行的进程的进程号

命令行参数

shell 脚本允许在命令行上输入选项和参数。

shell 脚本可以从命令行读入最多 10 个参数。它们被自动地置入“位置变量”当中。这些位置变量将在脚本中被使用。

- \$0 : 命令行上所键入的脚本文件名
- \$1,\$2,...,\$9 : 脚本文件名之后的第 1 到第 9 个参数
- \$# : 命令行参数的个数
- \$@ : 所有命令行参数
(每个参数以引号形式保存)
- \$* : 所有命令行参数
(保存在单一的字符串中)
- \$? : 最后一个命令的退出状态 (返回码)
- \$\$: 正在执行的进程的进程号

改进脚本：接受用户输入

- 前述 bigfile 脚本的局限：固定显示 3 个子目录
- 希望改成由用户灵活决定
- 采用 read 命令读入 shell 变量
- 改进后的程序如下：

bigfile2:

```
echo "The Numbers ?\c"  
read NUM  
ls -l ~ | sort -k 5 -n -r | head -n $NUM
```

改进脚本：接受用户输入

- 前述 bigfile 脚本的局限：固定显示 3 个子目录
- 希望改成由用户灵活决定
- 采用 read 命令读入 shell 变量
- 改进后的程序如下：

bigfile2:

```
echo "The Numbers ?\c"  
read NUM  
ls -l ~ | sort -k 5 -n -r | head -n $NUM
```

改进脚本：接受用户输入

- 前述 bigfile 脚本的局限：固定显示 3 个子目录
- 希望改成由用户灵活决定
- 采用 read 命令读入 shell 变量
- 改进后的程序如下：

bigfile2:

```
echo "The Numbers ?\c"  
read NUM  
ls -l ~ | sort -k 5 -n -r | head -n $NUM
```

改进脚本：接受用户输入

- 前述 bigfile 脚本的局限：固定显示 3 个子目录
- 希望改成由用户灵活决定
- 采用 read 命令读入 shell 变量
- 改进后的程序如下：

bigfile2:

```
echo "The Numbers ?\c"  
read NUM  
ls -l ~ | sort -k 5 -n -r | head -n $NUM
```

改进脚本：接受命令行参数

- 局限：只能显示用户主目录下的磁盘使用情况
- 改进：希望改成由用户灵活指定目标路径
- 手段：接受命令行参数
- 改进后的程序如下：

bigfile3:

```
echo "The Numbers ?\c"  
read NUM  
ls -l $1 | sort -k 5 -n -r | head -n $NUM
```

改进脚本：接受命令行参数

- 局限：只能显示用户主目录下的磁盘使用情况
- 改进：希望改成由用户灵活指定目标路径
- 手段：接受命令行参数
- 改进后的程序如下：

bigfile3:

```
echo "The Numbers ?\c"  
read NUM  
ls -l $1 | sort -k 5 -n -r | head -n $NUM
```

改进脚本：接受命令行参数

- 局限：只能显示用户主目录下的磁盘使用情况
- 改进：希望改成由用户灵活指定目标路径
- 手段：接受命令行参数
- 改进后的程序如下：

bigfile3:

```
echo "The Numbers ?\c"  
read NUM  
ls -l $1 | sort -k 5 -n -r | head -n $NUM
```


改进脚本：接受命令行参数

- 局限：只能显示用户主目录下的磁盘使用情况
- 改进：希望改成由用户灵活指定目标路径
- 手段：接受命令行参数
- 改进后的程序如下：

bigfile3:

```
echo "The Numbers ?\c"  
read NUM  
ls -l $1 | sort -k 5 -n -r | head -n $NUM
```

改进脚本：接受多个命令行参数

- 局限：两种参数指定方式不同，使用不便
- 改进：希望统一由命令行中指定
- 手段：接受多个命令行参数
- 改进后的程序如下：

bigfile4:

```
ls -l $1 | sort -k 5 -n -r | head -n $2
```

改进脚本：接受多个命令行参数

- 局限：两种参数指定方式不同，使用不便
- 改进：希望统一由命令行中指定
- 手段：接受多个命令行参数
- 改进后的程序如下：

bigfile4:

```
ls -l $1 | sort -k 5 -n -r | head -n $2
```

改进脚本：接受多个命令行参数

- 局限：两种参数指定方式不同，使用不便
- 改进：希望统一由命令行中指定
- 手段：接受多个命令行参数
- 改进后的程序如下：

bigfile4:

```
ls -l $1 | sort -k 5 -n -r | head -n $2
```

改进脚本：接受多个命令行参数

- 局限：两种参数指定方式不同，使用不便
- 改进：希望统一由命令行中指定
- 手段：接受多个命令行参数
- 改进后的程序如下：

bigfile4:

```
ls -l $1 | sort -k 5 -n -r | head -n $2
```

继续改进

- 局限：缺少一个参数，程序不能正常运行
- 局限：缺少两个参数，程序的运行不合愿望
- 希望：提示正确用法
- 希望：合理的缺省参数替代

条件与测试

if-then 结构

```
if [ condition ]  
then  
    commands  
    ...  
    last-command  
fi
```

条件与测试

if-then-else 结构

```
if [ condition ]
then
    true-commands
    ...
    last-true-command
else
    false-commands
    ...
    last-false-command
fi
```


改进脚本：检查参数个数

- 首先检查命令行中参数的个数
- 如果符合要求，继续执行
- 否则给出用法提示，然后退出
- 改进后的程序如下：

bigfile5:

```
if [ $# = 2 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n $2
else
    echo This command needs two parameters.
    echo "Usage: $0 <dir> <num>"
fi
```

改进脚本：检查参数个数

- 首先检查命令行中参数的个数
- 如果符合要求，继续执行
- 否则给出用法提示，然后退出
- 改进后的程序如下：

bigfile5:

```
if [ $# = 2 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n $2
else
    echo This command needs two parameters.
    echo "Usage: $0 <dir> <num>"
fi
```

改进脚本：检查参数个数

- 首先检查命令行中参数的个数
- 如果符合要求，继续执行
- 否则给出用法提示，然后退出
- 改进后的程序如下：

bigfile5:

```
if [ $# = 2 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n $2
else
    echo This command needs two parameters.
    echo "Usage: $0 <dir> <num>"
fi
```

改进脚本：检查参数个数

- 首先检查命令行中参数的个数
- 如果符合要求，继续执行
- 否则给出用法提示，然后退出
- 改进后的程序如下：

bigfile5:

```
if [ $# = 2 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n $2
else
    echo This command needs two parameters.
    echo "Usage: $0 <dir> <num>"
fi
```

改进脚本：使用缺省参数

- 首先检查命令行中参数的个数
- 如果符合要求，继续执行
- 否则按照一般用户的惯例，补充参数
- 改进后的程序见 bigfile6:

```
if [ $# = 2 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n $2
elif [ $# = 1 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n 3
elif [ $# = 0 ]
then
    ls -l ~ | sort -k 5 -n -r | head -n 3
fi
```

改进脚本：使用缺省参数

- 首先检查命令行中参数的个数
- 如果符合要求，继续执行
- 否则按照一般用户的惯例，补充参数
- 改进后的程序见 bigfile6:

```
if [ $# = 2 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n $2
elif [ $# = 1 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n 3
elif [ $# = 0 ]
then
    ls -l ~ | sort -k 5 -n -r | head -n 3
fi
```

改进脚本：使用缺省参数

- 首先检查命令行中参数的个数
- 如果符合要求，继续执行
- 否则按照一般用户的惯例，补充参数
- 改进后的程序见 bigfile6:

```
if [ $# = 2 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n $2
elif [ $# = 1 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n 3
elif [ $# = 0 ]
then
    ls -l ~ | sort -k 5 -n -r | head -n 3
fi
```

改进脚本：使用缺省参数

- 首先检查命令行中参数的个数
- 如果符合要求，继续执行
- 否则按照一般用户的惯例，补充参数
- 改进后的程序见 bigfile6:

```
if [ $# = 2 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n $2
elif [ $# = 1 ]
then
    ls -l $1 | sort -k 5 -n -r | head -n 3
elif [ $# = 0 ]
then
    ls -l ~ | sort -k 5 -n -r | head -n 3
fi
```


参数替换

参数替换的一般格式：

```
${parameter:option_character word}
```

例：

```
${VAR:-foobar}
```

以下是参数替换的不同形式：

<code>\$var</code>	保存在变量中的值
<code>\${var}</code>	保存在变量中的值
<code>\${var:-string}</code>	变量的值非空，取变量值；若为空，取string值
<code>\${var:+string}</code>	变量的值非空，取string值；若为空，取空
<code>\${var:=string}</code>	变量的值非空，取变量值；若为空，取string值，
<code>\${var:?string}</code>	变量的值非空，取变量值；若为空，显示string

参数替换

参数替换的一般格式：

```
${parameter:option_character word}
```

例：

```
${VAR:-foobar}
```

以下是参数替换的不同形式：

<code>\$var</code>	保存在变量中的值
--------------------	----------

<code>\${var}</code>	保存在变量中的值
----------------------	----------

<code>\${var:-string}</code>	变量的值非空，取变量值；若为空，取string值
------------------------------	--------------------------

<code>\${var:+string}</code>	变量的值非空，取string值；若为空，取空
------------------------------	------------------------

<code>\${var:=string}</code>	变量的值非空，取变量值；若为空，取string值，
------------------------------	---------------------------

<code>\${var:?string}</code>	变量的值非空，取变量值；若为空，显示string
------------------------------	--------------------------

改进脚本：使用参数替换的形式

- `${1:~}` 路径名的替换形式
- `${2:-3}` 目录个数的替换形式
- 改进后的程序如下：

bigfile7:

```
DIR=${1:~}  
NUM=${2:-3}  
ls -l $DIR | sort -k 5 -n -r | head -n $NUM
```

bigfile8:

```
ls -l ${1:~} | sort -k 5 -n -r | head -n ${2:-3}
```

改进脚本：使用参数替换的形式

- `${1:~}` 路径名的替换形式
- `${2:-3}` 目录个数的替换形式
- 改进后的程序如下：

bigfile7:

```
DIR=${1:~}  
NUM=${2:-3}  
ls -l $DIR | sort -k 5 -n -r | head -n $NUM
```

bigfile8:

```
ls -l ${1:~} | sort -k 5 -n -r | head -n ${2:-3}
```

改进脚本：使用参数替换的形式

- `${1:~}` 路径名的替换形式
- `${2:-3}` 目录个数的替换形式
- 改进后的程序如下：

bigfile7:

```
DIR=${1:~}  
NUM=${2:-3}  
ls -l $DIR | sort -k 5 -n -r | head -n $NUM
```

bigfile8:

```
ls -l ${1:~} | sort -k 5 -n -r | head -n ${2:-3}
```

改进脚本：使用参数替换的形式

- `${1:-~}` 路径名的替换形式
- `${2:-3}` 目录个数的替换形式
- 改进后的程序如下：

bigfile7:

```
DIR=${1:-~}  
NUM=${2:-3}  
ls -l $DIR | sort -k 5 -n -r | head -n $NUM
```

bigfile8:

```
ls -l ${1:-~} | sort -k 5 -n -r | head -n ${2:-3}
```

改进脚本：预防错误

- 局限：用户指定的目录不存在时，程序出错
- 局限：用户指定的显示行数为负数时，没有提醒
- 改进：希望改成周到、友好的提醒
- 手段：判断、处理
- 改进后的程序如下：

改进脚本：预防错误

- 局限：用户指定的目录不存在时，程序出错
- 局限：用户指定的显示行数为负数时，没有提醒
- 改进：希望改成周到、友好的提醒
- 手段：判断、处理
- 改进后的程序如下：

改进脚本：预防错误

- 局限：用户指定的目录不存在时，程序出错
- 局限：用户指定的显示行数为负数时，没有提醒
- 改进：希望改成周到、友好的提醒
- 手段：判断、处理
- 改进后的程序如下：

改进脚本：预防错误

- 局限：用户指定的目录不存在时，程序出错
- 局限：用户指定的显示行数为负数时，没有提醒
- 改进：希望改成周到、友好的提醒
- 手段：判断、处理
- 改进后的程序如下：

改进脚本：预防错误

- 局限：用户指定的目录不存在时，程序出错
- 局限：用户指定的显示行数为负数时，没有提醒
- 改进：希望改成周到、友好的提醒
- 手段：判断、处理
- 改进后的程序如下：

bigfile10:

```
DIR=${1:-~}
NUM=${2:-3}
if [ ! -d $DIR ]
then
    echo The path you specified does not exist. Please che
elif [ ! $NUM -gt 0 ]
then
    echo The Number should be grater than 0.
else
    ls -l $DIR | sort -k 5 -n -r | head -n $NUM
fi
```

test 命令详解

shell 脚本的控制结构，离不开条件测试，test 命令是最重要的测试手段。

前面 if-then 结构中的 [condition] 实际上就是 test 命令的另一种简写形式。

```
if [ "$VAR" = value ]
```

等同于：

```
if test "$VAR" = value
```

test 命令的各种判断形式：

1. 数值

-eq	-gt	-lt
-ne	-ge	-le

2. 字符串

=	!=	-n	-z
---	----	----	----

3. 文件

-r	-w	-s	-f	-d
----	----	----	----	----

4. 逻辑

-a	-o	!
----	----	---

test 命令的各种判断形式：

1. 数值

-eq	-gt	-lt
-ne	-ge	-le

2. 字符串

=	!=	-n	-z
---	----	----	----

3. 文件

-r	-w	-s	-f	-d
----	----	----	----	----

4. 逻辑

-a	-o	!
----	----	---

test 命令的各种判断形式：

1. 数值

-eq	-gt	-lt
-ne	-ge	-le

2. 字符串

=	!=	-n	-z
---	----	----	----

3. 文件

-r	-w	-s	-f	-d
----	----	----	----	----

4. 逻辑

-a	-o	!
----	----	---

test 命令的各种判断形式：

1. 数值

-eq	-gt	-lt
-ne	-ge	-le

2. 字符串

=	!=	-n	-z
---	----	----	----

3. 文件

-r	-w	-s	-f	-d
----	----	----	----	----

4. 逻辑

-a	-o	!
----	----	---

注意：

```
if test -f $1
```

也可写成：

```
if [ -f $1 ]
```

脚本范例程序的最终版

```
#!/bin/sh
# This script print the subdirectories which occupy more d
# Written by Someone Somebody
# Last modified on 2011-11-11 at 11:11:11
# Usage:
#     ./bigfile.sh  subdircitory  number
#
DIR=${1:-~}
NUM=${2:-3}
if [ ! -d $DIR ]
then
    echo The path you specified does not exist. Please che
elif [ ! $NUM -gt 0 ]
then
    echo The Number should be grater than 0.
```

shell 编程的更多内容

表达式运算命令 `expr`

shell 变量只能存放字符串类型的数据。
考虑：

```
x=1  
y=2  
echo $x, $y, $x + $y
```

会得到怎样的结果？

算术运算需要专门的命令：`expr`。应用的例子为：

```
expr 1 + 2  
expr $x + $y  
echo `expr $x + $y`
```

shell 编程的更多内容

表达式运算命令 `expr`

shell 变量只能存放字符串类型的数据。
考虑：

```
x=1  
y=2  
echo $x, $y, $x + $y
```

会得到怎样的结果？
算术运算需要专门的命令：`expr`。应用的例子为：

```
expr 1 + 2  
expr $x + $y  
echo `expr $x + $y`
```

expr 命令支持算术运算和关系运算。

1. 算术运算

expr 只能进行整数运算，五种运算为：

加：+，减：-，乘：*，除：/，取余：\%

2. 关系运算

关系运算的结果：0 表示“假”；1 表示“真”。

关系运算符号，是一目了然的。

= != \< \<= \> \>=

赋值命令 let

举例说明：

```
x=100
let x=x+1
echo $x      <-- 101
let y=x*2
echo $y      <-- 202
```

注意：let 是 bash 内部的命令，sh 并不支持。

赋值命令 `let`
举例说明：

```
x=100
let x=x+1
echo $x      <-- 101
let y=x*2
echo $y      <-- 202
```

注意：`let` 是 `bash` 内部的命令，`sh` 并不支持。

例：

在某个目录下，创建一系列子目录，每个子目录的名字，由一个固定的字符串开始，后面跟着一个自动递增的数字。比如：stu1, stu2, stu3, ..., stu40.

如何实现？

while 循环：while-do-done 结构

```
while [ condition ]  
do  
    commands  
    ...  
    last-command  
done
```

until 循环：until-do-done 结构

```
until [ condition ]  
do  
    commands  
    ...  
    last-command  
done
```

连续创建一系列子目录的 shell 程序:

```
n=1
max=40
while [ "$n" -le "$max" ]
do
    mkdir "stu$n"
    n=`expr "$n" + 1`
done
```

如果 shell 程序只在 bash 下执行，不必考虑 sh 的兼容性，还可以将

```
n=`expr "$n" + 1`
```

改为：

```
let n=n+1
```

循环结构

for 循环：for-in-done 结构

```
for variable in list-of-values
do
    commands
    ...
    last-command
done
```

工作目录下的一批文件改名：

report ⇒ David_report

plan ⇒ David_plan

novel ⇒ David_novel

使用 `mv * David_*` 这样的命令，是错误的。

正确的方式，是使用一个脚本：

```
for i in *
do
    mv -v $i David_$i
done
```

这个脚本的缺陷是：执行之后，会把自己的文件名也改掉。

工作目录下的一批文件改名：

report $\implies$ David_report

plan $\implies$ David_plan

novel $\implies$ David_novel

使用 `mv * David_*` 这样的命令，是错误的。

正确的方式，是使用一个脚本：

```
for i in *  
do  
    mv -v $i David_$i  
done
```

这个脚本的缺陷是：执行之后，会把自己的文件名也改掉。

工作目录下的一批文件改名：

report $\implies$ David_report

plan $\implies$ David_plan

novel $\implies$ David_novel

使用 `mv * David_*` 这样的命令，是错误的。

正确的方式，是使用一个脚本：

```
for i in *  
do  
    mv -v $i David_$i  
done
```

这个脚本的缺陷是：执行之后，会把自己的文件名也改掉。

使用一个命令行完成这个任务：

```
for i in *; do mv David_$i; done
```

改进：
脚本本身名字不改；

```
for i in *
do
    if [ ./$i != $0 ]
    then
        mv -v $i David_$i
    fi
done
```

这个脚本的缺陷是：再次执行，会把已经添加前缀的文件名继续添加前缀。

改进：
脚本本身名字不改；

```
for i in *  
do  
    if [ ./ $i != $0 ]  
    then  
        mv -v $i David_$i  
    fi  
done
```

这个脚本的缺陷是：再次执行，会把已经添加前缀的文件名继续添加前缀。

改进：
脚本本身名字不改；
已经有前缀的文件不再添加

```
for i in *  
do  
    if [ ./$i != $0 -a `echo $i | sed -e 's/David_//'\` =  
    then  
        mv -v $i David_$i  
    fi  
done
```

使用 for 循环重写批量建立子目录的 shell 脚本

```
#!/bin/bash
for n in {1..40}
do
    mkdir "stu$n"
done
```

这个脚本只能使用 bash 执行。

兼容 sh 的版本：

```
#!/bin/sh
for n in `seq 1 40`
do
    mkdir "stu$n"
done
```

这个脚本兼容 sh 和 bash 。

shell 脚本的调试

长的，或控制结构复杂的脚本，很容易包含错误。需要有可行的排错手段，用来调试脚本。

shell 有三个常用选项，可以帮助 shell 脚本的调试。

名称	选项	功能
不执行	-n	读入命令行但不执行
冗余	-v	显示读入的行
执行跟踪	-x	执行时显示命令和参数

启用调试选项的方法

1. 调用时启用

```
sh -x myscript
```

2. 在脚本的首行启用

脚本文件中，用“#”开始的行，都被视为注释行。但第一行的头两个字符为“#!”时，另有含义。

脚本文件的第一行为：

```
#!/bin/sh
```

表示该脚本由 /bin/sh 解释执行。

```
#!/bin/sh -x
```

增加了执行选项。

3. 在脚本中间用 set 命令启用

```
set -x      (启用)
```

```
set +x      (关闭)
```

```
set -        (关闭所有已打开的调试选项)
```

启用调试选项的方法

1. 调用时启用

```
sh -x myscript
```

2. 在脚本的首行启用

脚本文件中，用“#”开始的行，都被视为注释行。但第一行的头两个字符为“#!”时，另有含义。

脚本文件的第一行为：

```
#!/bin/sh
```

表示该脚本由 /bin/sh 解释执行。

```
#!/bin/sh -x
```

增加了执行选项。

3. 在脚本中间用 set 命令启用

```
set -x      (启用)
```

```
set +x      (关闭)
```

```
set -       (关闭所有已打开的调试选项)
```

启用调试选项的方法

1. 调用时启用

```
sh -x myscript
```

2. 在脚本的首行启用

脚本文件中，用“#”开始的行，都被视为注释行。但第一行的头两个字符为“#!”时，另有含义。

脚本文件的第一行为：

```
#!/bin/sh
```

表示该脚本由 /bin/sh 解释执行。

```
#!/bin/sh -x
```

增加了执行选项。

3. 在脚本中间用 set 命令启用

```
set -x      (启用)
```

```
set +x      (关闭)
```

```
set -       (关闭所有已打开的调试选项)
```

Part XIII

流编辑器 sed

流编辑器 sed 的简单介绍

通过 shell 编程，可以将 Linux 的各种命令组织起来，完成复杂的任务。

必须有更加强大的工具，提供 shell 编程的基础。

流编辑器（Stream Editor）是一种基于正则表达式的，非交互式的编辑器。

sed 基础

一. sed 的基本工作方式

对于输入文件的每一行，执行一系列的 sed 命令。这些 sed 命令被称为 sed 脚本。

sed 脚本针对输入文件，或者输入流的每一个文本行进行处理。

二. sed 命令的语法 `sed 'script' files`

缺省情况下，处理之后的结果提供标准设备输出，也就是显示在屏幕上。

当命令后面没有跟上文件名时，处理的对象就是标准输入。

sed 也可以接受通过管道送来的字符流。

第一个例子

假设有一个文件，名字为：tel 其中当内容，是一些城市的长途电话直拨号码前缀。内容大体为：

```
1    nj    025
2    gz    020
3    sh    021
4    bj    010
5    wh    027
6    cd    028
```

用 sed 命令，把区号之前的“0”换成“+86”。

形式一：命令行

```
sed 's/\<0/+86/' tel
```


形式二：sed 脚本文件

把 `s/\<0/+86/` 写到一个文件之中，比如文件名为：`inter`
在命令行上输入命令：

```
sed -f inter tel
```

形式三：使用管道

```
cat tel | sed 's/\<0/+86/'
```

或者：

```
cat tel | sed -f inter
```

把区号字段，和城市名字段对调，应当怎样实现？

形式三：使用管道

```
cat tel | sed 's/\<0/+86/'
```

或者：

```
cat tel | sed -f inter
```

把区号字段，和城市名字段对调，应当怎样实现？

sed 脚本的执行与模式空间

- ❶ 从输入文件中读取一行
- ❷ 把这一行复制出一个副本，放在缓冲区中
- ❸ 对于缓冲区中的副本，应用 sed 脚本。
- ❹ 如果 sed 脚本有多行，这些脚本行将依次被执行
- ❺ 输入文件的一行处理完毕，再取输入文件的下一行，做同样处理。

上面提到的存放文本行副本的缓冲区，就是“模式空间”。
sed 的一切操作，都是在模式空间中进行的。

sed 脚本的执行与模式空间

- ❶ 从输入文件中读取一行
- ❷ 把这一行复制出一个副本，放在缓冲区中
- ❸ 对于缓冲区中的副本，应用 sed 脚本。
- ❹ 如果 sed 脚本有多行，这些脚本行将依次被执行
- ❺ 输入文件的一行处理完毕，再取输入文件的下一行，做同样处理。

上面提到的存放文本行副本的缓冲区，就是“模式空间”。
sed 的一切操作，都是在模式空间中进行的。

sed 脚本

sed 脚本，是由一行行的 sed 命令组成的。
只有一个单行的 sed 脚本，是常见的情况。

语法

最一般的 sed 脚本命令形式：

```
[address] command
```

方括号之内的行地址是可选的。

个别只能用于单行地址的 sed 脚本命令，表示成：

```
[line-address] command
```

command 部分可以有 25 种不同的命令。只需要掌握其中很少的一部分，就可以处理大多数的情况。

sed 脚本

sed 脚本，是由一行行的 sed 命令组成的。
只有一个单行的 sed 脚本，是常见的情况。

语法

最一般的 sed 脚本命令形式：

```
[address] command
```

方括号之内的行地址是可选的。

个别只能用于单行地址的 sed 脚本命令，表示成：

```
[line-address] command
```

command 部分可以有 25 种不同的命令。只需要掌握其中很少的一部分，就可以处理大多数的情况。

sed 脚本

sed 脚本，是由一行行的 sed 命令组成的。
只有一个单行的 sed 脚本，是常见的情况。

语法

最一般的 sed 脚本命令形式：

```
[address] command
```

方括号之内的行地址是可选的。

个别只能用于单行地址的 sed 脚本命令，表示成：

```
[line-address] command
```

command 部分可以有 25 种不同的命令。只需要掌握其中很少的一部分，就可以处理大多数的情况。

常用的 sed 脚本命令

1. 替换命令 s

用法：

```
[address]s/pattern/replacement/flags
```

我们已经用过这个命令，例如：

```
s/oldstring/newstring/
```

这条命令的后面有几种可以选择的标志，用来修饰替换方式。

- ❶ n：1 到 512 之间的一个数字，表示对指定模式中第 n 次匹配处进行替换。
- ❷ g：对模式空间中的所有符合匹配模式的内容进行替换。
- ❸ p：显示模式空间内容
- ❹ w file：将模式空间的内容写到文件 file 中。

常用的 sed 脚本命令

1. 替换命令 s

用法：

```
[address]s/pattern/replacement/flags
```

我们已经用过这个命令，例如：

```
s/oldstring/newstring/
```

这条命令的后面有几种可以选择的标志，用来修饰替换方式。

- ❶ n：1 到 512 之间的一个数字，表示对指定模式中第 n 次匹配处进行替换。
- ❷ g：对模式空间中的所有符合匹配模式的内容进行替换。
- ❸ p：显示模式空间内容
- ❹ w file：将模式空间的内容写到文件 file 中。

常用的 sed 脚本命令

1. 替换命令 s

用法：

```
[address]s/pattern/replacement/flags
```

我们已经用过这个命令，例如：

```
s/oldstring/newstring/
```

这条命令的后面有几种可以选择的标志，用来修饰替换方式。

- ❶ n：1 到 512 之间的一个数字，表示对指定模式中第 n 次匹配处进行替换。
- ❷ g：对模式空间中的所有符合匹配模式的内容进行替换。
- ❸ p：显示模式空间内容
- ❹ w file：将模式空间的内容写到文件 file 中。

2. 显示命令 p 假设有一个记录水果价格的文件，名为：
fruite_prices.txt，内容为：

Fruit	Price/lbs
Banana	0.89
Paech	0.79
Kiwi	1.50
Pineapple	1.29
Apple	0.99
Mango	2.20

要求列出价格低于 1 美元/磅的水果列表。

观察正则表达式：`/ 0\.[0-9][0-9]$/`

2. 显示命令 p 假设有一个记录水果价格的文件，名为：
fruite_prices.txt，内容为：

Fruit	Price/lbs
Banana	0.89
Paech	0.79
Kiwi	1.50
Pineapple	1.29
Apple	0.99
Mango	2.20

要求列出价格低于 1 美元/磅的水果列表。
观察正则表达式：`/ 0\.[0-9][0-9]$/`

使用命令：

```
sed '/ 0\.[0-9][0-9]$/p' fruite_prices.txt
```

结果怎样？

再使用命令：

```
sed -n '/ 0\.[0-9][0-9]$/p' fruite_prices.txt
```

结果怎样？

3. 删除命令 d

芒果已售完，希望从列表中删去， sed 脚本怎样写？

```
sed '/^[Mm]ango/d' fruited_pricea.txt
```

删除的动作，只在模式空间之内发生，不会影响原有的文件。

3. 删除命令 d

芒果已售完，希望从列表中删去， sed 脚本怎样写？

```
sed '/^[Mm]ango/d' fruited_pricea.txt
```

删除的动作，只在模式空间之内发生，不会影响原有的文件。

4. 追加、插入、更改 (a, i, c)

```
[line-address] a \  
text
```

```
[line-address] i \  
text
```

```
[line-address] c \  
text
```

这些命令分别把所提供的文本，放置于模式空间的文本行之后、之前，或者取代它。

5. 列表命令 l

将模式空间内的字符列出。

其他的 sed 命令：

y 转换命令

= 打印行号

n 下一行

r 读文件

w 写文件

q 退出

Part XIV

awk

基于模式匹配的文本处理程序 awk

awk 的名字来源于它的三个作者：

Alfred Aho

Peter Weinberger

Brian Kernighan

工作方式像 sed 和 C 语言的混合。

awk 的基本语法

```
awk 'script' files
```

或者：

```
awk -f scriptfile files
```

awk 的工作方式

对于输入文件的每一行，执行一系列的 awk 命令。这些 awk 命令被称为 awk 脚本。

awk 脚本针对输入文件，或者输入流的每一个文本行进行处理。缺省情况下，处理之后的结果提供标准设备输出，也就是显示在屏幕上。

当命令后面没有跟上文件名时，处理的对象就是标准输入。

awk 也可以接受通过管道送来的字符流。

模式与动作

典型的 awk 命令包括模式和动作：

```
/patter/ { actions }
```

例如：

```
awk '{print}' telbook
```

```
awk '/[a-z]j/ {print}' telbook
```


一个例子

假设有一个文件，名字为：grade 其中当内容是一个班级学生的考试成绩。我们用一个 awk 的脚本来计算学生各门课的平均分，全班各门课的平均分，以及总的平均分。

name	math	english	maogai
Tom	82	76	97
David	76	84	62
Linda	79	95	70
Adam	83	78	75
Bob	67	87	66
Eva	96	63	71

awk 脚本文件

```
NR == 1 {    printf "%s\t%s\n", $0 , "average" }
NR != 1 {    math += $2;
              english += $3;
              maogai += $4;
              avg = ( $2 + $3 + $4 ) / 3;
              sumavg += avg;
              printf "%s\t%3.2f\n", $0 , avg }
END {        num = NR - 1;
              printf "average\t%3.2f\t%3.2f\t%3.2f\t%3.2f\n",
              math/num, english/num, maogai/num, sumavg/num }
```

Part XV

Linux 环境下的 C 语言编程

最简单的过程

- 使用编辑器创建源程序文件
vi hello.c
- “转换” 成可执行的程序代码
gcc hello.c
- 得到的结果是，当前目录下的文件
a.out
- 执行这个程序
./a.out

最简单的过程

- 使用编辑器创建源程序文件
vi hello.c
- “转换” 成可执行的程序代码
gcc hello.c
- 得到的结果是，当前目录下的文件
a.out
- 执行这个程序
./a.out

最简单的过程

- 使用编辑器创建源程序文件
vi hello.c
- “转换” 成可执行的程序代码
gcc hello.c
- 得到的结果是，当前目录下的文件
a.out
- 执行这个程序
./a.out

最简单的过程

- 使用编辑器创建源程序文件
vi hello.c
- “转换” 成可执行的程序代码
gcc hello.c
- 得到的结果是，当前目录下的文件
a.out
- 执行这个程序
./a.out

指定可执行程序的名字

```
gcc -o hello hello.c
```

```
gcc hello.c -o hello
```


gcc 的编译过程

- 预处理
- 编译
- 汇编
- 链接
- 优化

gcc 的编译过程

- 预处理
- 编译
- 汇编
- 链接
- 优化

gcc 的编译过程

- 预处理
- 编译
- 汇编
- 链接
- 优化

gcc 的编译过程

- 预处理
- 编译
- 汇编
- 链接
- 优化

gcc 的编译过程

- 预处理
- 编译
- 汇编
- 链接
- 优化

gcc 编译过程的示意图

```
x.c --- x.i --- x.s --- x.o
                        \
                        ====> hello
                        /
y.c --- y.i --- y.s --- y.o
```

仅仅预处理

```
gcc -E hello.c -o hello.i
```

仅仅生成汇编语言

```
gcc -S hello.i
```

```
gcc -S hello.c
```


仅仅编译出目标文件（不执行链接操作）

```
gcc -c hello.s
```

```
gcc -c hello.i
```

```
gcc -c hello.c
```

生成最终的可执行文件

```
gcc hello.o
```

```
gcc hello.i
```

```
gcc hello.s
```

```
gcc hello.c
```

指定可执行程序的名字

```
gcc -o hello hello.o
```

```
gcc -o hello hello.i
```

```
gcc -o hello hello.s
```

```
gcc -o hello hello.c
```